AI-ENABLED COCONUT LAND UTILIZATION, WCLWD SEVERITY ASSESSMENT, GERMPLASM IDENTIFICATION AND COMMUNITY INTELLIGENCE ADVISORY SYSTEM

INDIVIDUAL THESIS

Randeniya A.A.S.L.B.R.P.W.R.C

IT22236500

AI-Based Image Classification System for Coconut Germplasm Identification

R26-IT-019

BSc (Hons) degree in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

[Month Year]

AI-ENABLED COCONUT LAND UTILIZATION, WCLWD SEVERITY ASSESSMENT, GERMPLASM IDENTIFICATION AND COMMUNITY INTELLIGENCE ADVISORY SYSTEM

INDIVIDUAL THESIS

Randeniya A.A.S.L.B.R.P.W.R.C

IT22236500

AI-Based Image Classification System for Coconut Germplasm Identification

R26-IT-019

Dissertation submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honours in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

[Month Year]

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Randeniya A.A.S.L.B.R.P.W.R.C

Student ID: IT2223500

Signature:  Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor:		
Co-supervisor:		

ABSTRACT

Image-based coconut germplasm identification could be a solution to the problem of finding a proper way to identify varieties with less access to experts. However, it can be difficult to develop a model due to limited labeled images and visually similar categories of coconut. This research describes a deep-learning-based image classification model for classifying Dwarf, King Coconut, and Tall coconut categories, as well as visual explanations of the predicted categories. This dataset consisted of 428 images; 118 Dwarf, 67 King Coconut, and 243 Tall. A class-wise random split resulted in 341 training and 87 validation images. Images were resized to 224 pixels, normalised with Image Net statistics and augmented during training. The model consisted of an Image Net-pretrained Efficient Net-B0 as a feature extractor, a 512 dimensional embedding layer, and a Prototypical Network with cosine similarity metric. To train, we used three-way, eight-shot, five-query episodes over 40 epochs. Grad-CAM was adapted for the prototype similarity score in order to visualise the regions of the image that contributed to the choice of prediction. The best checkpoint for the model achieved a validation accuracy of 98.67% at epoch 27. Further analysis on 50 episodes sampled from this validation set revealed 724 correct predictions out of 750 query predictions, episodic accuracy=96.53%. As there was repeat sampling of images, these predictions are not representative of an independent test-set evaluation. The contribution is the classification pipeline, the episodic model training and evaluation, model export, and the Grad-CAM implementation. The results showed a promising performance in the conditions considered in this work. Further testing on other locations and conditions of acquisition and validation of the visual explanations by an expert in this regard is needed for broader use.

Keywords: Coconut germplasm detection, Few shot learning, Prototypical Network, Efficient Net-B0, Grad-CAM.

ACKNOWLEDGEMENTS

I want to express my sincere gratitude to my research supervisor, Prof. Sanvitha Kasthuriarchchi, for her continuous guidance, valuable suggestions, and constructive feedback throughout the development of this research. Her expertise, encouragement, and academic support were instrumental in shaping the direction and quality of this research work. I would also like to extend my sincere appreciation to my co-supervisor, Ms. Karthiga Rajendran, for her academic guidance, helpful comments, and continuous encouragement throughout the research planning process, which greatly improved the quality of this proposal. My sincere thanks also go to the external supervisor, Mr. Oshan Thilakarathne from the Coconut Research Institute (CRI), for providing valuable domain knowledge, technical insights, and support in understanding coconut research practices and related field information. I would also like to express my appreciation to Mr. Samitha Vidhanarachchi for his guidance, technical support, and valuable assistance provided during the development of this research proposal. Furthermore, I would like to thank the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic environment, facilities, and resources necessary to conduct this research. Finally, I would like to express my heartfelt gratitude to my family and friends for their constant encouragement, motivation, and support throughout this research project.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 3

1.4 Research gap 4

1.5 Aim and objectives 7

1.5.1 Research aim 7

1.5.2 Specific objectives 7

1.6 Research questions or hypotheses 7

1.7 Scope limitations and assumptions 8

1.7.1 Scope 8

1.7.2 Limitations 9

1.7.3 Assumptions 9

1.8 Individual contribution 10

1.9 Thesis structure 11

2 LITERATURE REVIEW AND RELATED WORK 13

2.1 Review method and scope 13

2.2 Domain and theoretical foundations 14

2.3 Related Research on Deep Learning for Plant Image Classification 15

2.4 Existing systems, products and techniques 17

2.4.1 Existing plant identification systems and products 17

2.4.2 Transfer learning and EfficientNet-based feature extraction 17

2.4.3 Few-shot learning and prototype-based classification 18

2.4.4 Visual explanation techniques 19
2.5 Gap synthesis and conceptual framework 20
2.5.1 Gap synthesis 21
2.5.2 Conceptual framework 23
3 RESEARCH METHODOLOGY 24
3.1 Research design 24
3.2 Data participants or study materials 26
3.2.1 Dataset source and composition 26
3.2.2 Training and validation partition 27
3.2.3 Maintain and seek information for study 27
3.2.4 Data coverage and limitations 28
3.3 Experimental or study procedure 28
3.3.1 Image preprocessing and augmentation. 29
3.3.2 Episodic sampling 29
3.3.3 Feature extraction and prototype-based classification 29
3.3.4 Model training and checkpoint selection 31
3.3.5 Post-training episodic evaluation 32
3.3.6 Fixed-prototype inference and visual inspection 32
3.3.7 Export of inference artefacts 33
3.4 Measures and evaluation metrics 34
3.5 Data analysis 35
3.6 Ethics privacy security and responsible AI 36
3.7 Reproducibility and research artefacts 37
4 4 PRODUCT AND SOLUTION DESIGN 39
4.1.1 Requirements 39
4.2 Component architecture within the complete solution 41

4.3 Design alternatives and decisions 43
4.4 Detailed design 44
4.4.1 Preprocessing and embedding generation 44
4.4.2 Prototype comparison and visualization 44
4.4.3 Service coordination and artefact consistency 44
4.5 Standards and professional practice 45
5 IMPLEMENTATION INTEGRATION AND TESTING 46
5.1 Development environment and technology stack 46
5.2 Component implementation 47
5.2.1 Embedding model and episodic training 47
5.2.2 Prediction of fixed-prototype 48
5.2.3 Grad-CAM and model export 48
5.3 Integration with the other components 48
5.4 Verification and software testing 49
5.5 Deployment and operation 50
6 6 RESULTS AND VALIDATION 52
6.1 6.1 Research results 52
6.1.1 Training Behavior 52
6.1.2 Classification Performance 52
6.1.3 Scope of the Evaluation 54
6.2 6.2 Product and technical results 54
6.2.1 Classification Output 55
6.2.2 Grad-CAM Visualization Output 55
6.2.3 Model Export and Technical Verification 56
6.3 User or stakeholder validation 57
6.4 Integration results 57

6.5 Summary of findings 58
7 DISCUSSION 59
7.1 Interpretation of findings 59
7.2 Comparison with literature baselines and existing solutions 60
7.3 Research contribution 60
7.4 Product contribution 61
7.5 Limitations and threats to validity 62
7.5.1 Data set and generalizations 62
7.5.2 Evaluation Procedure 62
7.5.3 Explanation and Practical Validation 62
7.6 Implications 63
8 COMMERCIALISATION AND SUSTAINABILITY 65
8.1 Value Proposition and Target Users 65
8.2 Market and Competitive Position 66
8.3 Business or Sustainability Model 66
8.4 Scalability, Intellectual Property and Compliance 67
8.4.1 Scalability 67
8.4.2 Intellectual Property 67
8.4.3 Compliance and Responsible Operation 68
8.5 Alignment with Sustainable Development Goals 68
9 CONCLUSION AND FUTURE DIRECTIONS 70
9.1 Conclusion 70
9.2 Achievement of objectives 70
9.3 Individual research and product contributions 71
9.4 Future directions 72
10 REFERENCES 74

11 GLOSSARY 76

12 APPENDICES 78

12.1 Appendix A Research instruments or extended results 78

12.1.1 A.1 Per-Epoch Training Log..... 78

12.2 Appendix B Academic integrity and AI use disclosure**Error! Bookmark not defined.**

LIST OF FIGURES

Figure 3.1:Development and evaluation workflow for the coconut germplasm classification component, adapted from [14]. 24
Figure 3.2: Episodic training and fixed-prototype inference procedure for the coconut classification component. 32
Figure 4.1 : Architecture of the coconut germplasm classification component within the integrated platform. 41
Figure 6.1:Training loss, training query accuracy, and validation query accuracy across 40 epochs. 52
Figure 6.2: Confusion matrix for query predictions across 50 validation episodes... 54
Figure 6.3: Example image classified as King Coconut by the implemented component. 55
Figure 6.4: Original image, Grad-CAM heatmap, and overlay for the King Coconut prediction. 56

LIST OF TABLES

Table 1.1: Comparison of selected previous studies and the present research 5
Table 2.1Comparison of Selected Deep Learning Studies for Plant Image Analysis 16
Table 2.2Comparison of Existing Products and Techniques Relevant to Coconut Image Classification..... 19
Table 2.3Synthesis of Literature Findings and Research Responses..... 22
Table 3.1:Coconut image dataset composition and training–validation split 27
Table 3.2:Experimental configuration for episodic model training and validation ... 31
Table 3.3: Research artefacts and their purposes..... 37
Table 4.1:Functional requirements of the coconut classification component..... 39
Table 4.2:Non-functional requirements of the coconut classification component 40
Table 4.3: Architectural elements of the classification component..... 42
Table 4.4:Design alternatives and selected approaches..... 43
Table 5.1: Development environment and technology stack..... 46
Table 5.2: Test cases for the coconut classification component..... 49
Table 6.1: Classification performance across 50 validation episodes 53
Table 8.1: Target users and proposed value of the component..... 65
Table 9.1: Summary of objective achievement..... 71
Table 12.1:Per-epoch training log..... 78

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CRI	Coconut Research Institute
CUDA	Compute Unified Device Architecture
Grad-CAM	Gradient-weighted Class Activation Mapping
ML	Machine Learning
ONNX	Open Neural Network Exchange
RGB	Red Green Blue
SLIIT	Sri Lanka Institute of Information Technology

1 INTRODUCTION

1.1 Overall project context

The main idea of the project is to create an AI-powered platform to help with coconut management in Sri Lanka. This project consolidates coconut land use analysis, Weligama coconut leaf wilt disease severity evaluation, coconut germplasm classification, and community intelligence advisory functions. Those components cater to various information requirements of coconut cultivation and can provide a connected decision-support platform for farmers, plantation managers, and researchers.

The function is expected to upload either field or aerial images according to the selected function and display related results. Land utilization analysis facilitates review of current cropping systems, as well as potential planting locations. Disease severity analysis generates image-based support to evaluate the visible signs. Germplasm identification recognizes coconut images and, based on the trained model, categorizes them into one of the categories. The advisory component offers guidance in English and Sinhala. Thus the project aims to make research outputs more accessible by developing research outputs more accessible through a common web application. In this overall solution, the work I did individually for this thesis was on the AI-based image classification part for coconut germplasm identification. The classification range of this part is limited to 3 annotated groups: Dwarf, King Coconut, and Tall. This component is a hybrid visual feature extractor pretrained with a prototypical network and enables inspection of the predicted image regions via Grad-CAM visualizations.

The connection between this individual component and the larger project draws on the delivery of variety-related information in conjunction with other outputs around coconut management. To be described as a coconut category, predicted characteristics can aid initial identification, and users will be able to know when further specialist input is required. Image predictions, however, do not ascertain genetic identity, indicate if the coconut is appropriate to the site, or drive any increased productivity. This thesis focuses on the germplasm classification aspect of the system, including its design, implementation, and evaluation. The other aspects are presented to provide a

comprehensive overview of the overall project and the context of the system, with which they are integrated. Detailed methods and results are beyond the scope of this thesis.

1.2 Individual component context

The individual component of this study aims to create an image classification component using AI for coconut germplasm identification. As a part of the integrated coconut management platform, the component classifies the uploaded image to any of the three predetermined categories (Dwarf, King Coconut, Tall). Expected by the system is a prediction of the selected category and a visual explanation of the regions of the input image that contributed most to the system prediction.

For this implementation, 428 images were used, consisting of 118 Dwarf images, 67 King Coconut images, and 243 Tall images. With these numbers, very few images and much unbalanced distribution among different classes, it was decided to use a pretrained feature extractor along with an episodic classifier. Efficient Net-B0 The feature extractor utilizes Efficient Net-B0, part of a family of models called Efficient Net, developed by Tan and Le [1], which explored the effects of model depth, width and input resolution to optimize both the efficiency and accuracy of CNNs. In this component, an Image Net-trained Efficient Net-B0 backbone pre-trained network is used to extract features from an image, which are then mapped into a 512-dimensional embedding.

The task classifier is based on the class-prototype idea from Prototypical Networks [2]. One prototype is defined per class based on the mean of the support image embeddings per class. The prototype network classifies a query image by determining the similarity between its embedding and the class prototypes, for which implementation details include a learnable scale parameter and cosine similarity. This can be seen as a variation of the base approach for the chosen coconut classes.

Training has a three-way, eight-shot, five-query episodic structure. An episode of training contains all three categories, with eight support images and five query images for each category. The images are split into training and validation sets with a class-wise split. Both have the same categories. So while the implementation adopts the one

used for few-shot learning [2], the current evaluation is on the three coconut categories, rather than classifying unseen categories.

The component makes use of the Gradient-weighted Class Activation Mapping (Grad-CAM) tool for aiding prediction interpretation [3]. Grad-CAM applies gradients to a particular output to draw attention to the areas of the image which significantly affect the output [3]. The class prototype similarity score for the selected class is used to generate gradients through the final convolutional layer. The resulting heat map is overlayed on top of the input image so that the user can examine the areas of the image which led to the predicted class.

For inference and integration, this component can set up trained model, file of class prototypes, class labels and descriptions as well as necessary pre-processing information. Its tested functionality only covers the three target categories and the available dataset. It thus provides a platform for easily accessible visual identification, while genetic confirmation, robust identification of unknown categories and identification of planting suitability are not tested.

1.3 Problem statement

This project deals with the Classification of Coconuts in three selected categories (Dwarf, King Coconut and Tall) by utilizing a small, imbalanced image dataset. The consumer part must give us the predicted category and visual information that lead us to inspect the prediction. Its goal is to help basic visual identification by using the integrated coconut management platform.

The data set available consists of 428 images, 118 Dwarf, 67 King Coconut and 243 Tall. The challenge in developing a system is to learn relevant differences between these classes from the examples provided, allowing for their uneven representation. Assessment will need to assess performance by class as overall accuracy can hide differences between classes.

An additional challenge is how to interpret the output. While a predicted category informs you what the model decided, it cannot tell you where in the image it was looking to make this decision. So this component needs to be able to check if the

decision is correlated to plausible coconut attributes, or more likely, not the background. Visual explanations can help with this.

The assessment should clarify what limits are placed on the demonstrated performance. Performance in the available validation subset does not demonstrate the model will work at other sites, under other image capture conditions or on other coconut classes not included in the validation subset. Since the validation subset is used for model selection, no independent test is carried out. Image based classification does not confirm genetic identity of a coconut plant.

The central research problem is therefore to develop and evaluate an image classification component that distinguishes the three selected coconut categories using limited image data, supports visual inspection of its predictions, and reports its performance within clearly defined evaluation limits.

1.4 Research gap

Existing literature forms the basis of plant identification by using morphological features to classify plants. In Sri Lankan coconuts, Ekanayake et al. described coconut types by their stature, stem form, crown size, nut type and breeding behavior [4]. The principles of their research do indicate the need to consider visual traits when classifying coconuts, but it does not assess an automated image classification system and some of the traits considered may need more than one image to be identified.

Deep learning has also been used to automate plant recognition. Lee et al. introduced Deep-Plant to learn from visual features of leaf images using a convolutional neural network [5]. Carranza-Rojas et al. used deep learning to classify herbarium specimens and studied transfer learning between image collections [6]. Although these studies relate to plant recognition, they did not test the recognition within the categories of Dwarf, King Coconut, and Tall as studied here and so their results are more methodological building blocks rather than validation of this component.

However, an additional issue is performance variation across various image capture settings. Mohanty et al. achieved high classification accuracy for plant diseases on a dataset of images captured in controlled conditions, then significantly lower accuracy on a dataset of images captured in different conditions [7]. While this task is different

from classifying coconut categories, this illustrates the need for application-specific assessment of system accuracy. In this case, the classification accuracy will have to be measured on the images at hand, and the generalizability to other locations, backgrounds, viewpoints, and lighting conditions is a separate consideration.

Prediction explanation adds another consideration. [3] The visualization methods show areas in the image influencing a chosen model output, which is useful. Visual explanation needs interpretation. [8] Adebayo et al. showed that visual methods can pass model and data randomization tests, but visual appearance alone is not indicative of the appropriateness of an explanation. They used these tests on saliency methods, which is a good reason not to use visualizations to confirm biological validity in our own experiments.

Thus, the gap this work aims to fill is the use and testing of pretrained feature extraction, prototype-based classification and visual explanations for the selected coconut categories with a small, imbalanced dataset. None of the published works tested the above combination with the 428-image dataset used in this project. This work presents an EfficientNet-B0-based Prototypical Network for classifying Dwarf, King Coconut, and Tall images along with Grad-CAM visualizations for further examination of predictions.

Table 1.1: Previous studies and their relevance to this research. The contribution is: Application-specific implementation and evaluation of relevant existing methods The work does not prove that the automation of coconut detection is not present in other work in the literature. Additional independent testing of other capture locations and conditions is required for extrapolation.

Table 1.1: Comparison of selected previous studies and the present research

Study	**Research focus**	**Approach**	**Relevance and distinction from the present research**
Ekanayake et al. [4]	Classification of coconut forms in Sri Lanka	Morphological characterization	Provides coconut classification evidence; does not evaluate

			automated identification from submitted images.
Lee et al. [5]	Plant identification from leaf images	CNN-based visual feature learning	Supports image-based plant recognition; does not evaluate the three coconut categories used in this study.
Carranza-Rojas et al. [6]	Automated identification of herbarium specimens	Deep learning and transfer learning	Investigates recognition and transfer between image collections; uses a different image domain from the present dataset.
Mohanty et al. [7]	Image-based plant disease detection	Deep convolutional neural networks	Highlights performance changes across capture conditions; evaluates disease classification rather than coconut category identification.
Adebayo et al. [8]	Evaluation of saliency explanations	Model and data randomization checks	Supports careful assessment of explanation maps; does not develop a coconut identification system.
Present research	Classification of Dwarf, King Coconut, and Tall	EfficientNet-B0, Prototypical Network, and Grad-CAM	Evaluates classification and generates visual explanations using 428 images; independent field testing remains outstanding.

1.5 Aim and objectives

1.5.1 Research aim

To develop and evaluate an AI-based image classification component that distinguishes Dwarf, King Coconut, and Tall coconut categories using a limited image dataset and provides visual explanations to support preliminary identification.

1.5.2 Specific objectives

1. Prepare the coconut image dataset by organizing a set of labelled images in the chosen categories on the coconut image dataset and performing the necessary data pre-processing, data augmentation and data train-validation split.
2. Build a classification system using a prototype method and an EfficientNet-B0 feature extractor that's been pretrained on ImageNet, then use episodic training to classify the coconut categories you've selected.
3. Evaluate classification performance using episodic accuracy, class-specific precision, recall, F1-score, and a confusion matrix, while documenting the limitations of the validation-based evaluation.
4. Use Grad-CAM visualizations for the top three predictions of the group to visualize the image areas influencing the predictions and qualitatively view the relevant features and foreground attention.
5. Integrate the classification component into the project's web application to allow users to submit coconut images and view predicted categories and associated visual explanations.
6. Identify limitations and opportunities for improvement, including independent testing, broader field data collection, and evaluation of coconut categories outside the current classification scope.

1.6 Research questions or hypotheses

This study is guided by the following research questions:

1. How effectively can an EfficientNet-B0-based Prototypical Network distinguish Dwarf, King Coconut, and Tall images using the available limited and imbalanced dataset?

2. How does classification performance vary across the three categories in terms of precision, recall, F1-score, and the confusion matrix?
3. Which image regions are highlighted by Grad-CAM for correct and incorrect predictions, and what do these visualizations reveal about attention to coconut features and background areas?
4. How can the trained classification model and its visual explanations be integrated into the web application to support image submission and presentation of predictions?
5. What limitations of the dataset and evaluation procedure affect the interpretation of the results and the component's potential use under broader field conditions?

1.7 Scope limitations and assumptions

1.7.1 Scope

This individual project aims to implement and test an image classification module for three coconut varieties, namely Dwarf, King Coconut and Tall. The report describes the preparation of datasets, image processing and data augmentation, model training, testing of classification accuracy, creating visual explanations and deployment of the application.

This implementation employs 428 labelled images and integrates an Efficient Net-B0 feature extractor [1] (pre-trained on Image Net) with a classification method inspired by Prototypical Networks [2]. In addition, Grad-CAM is added in order to produce heatmaps visualizing image regions influencing class predictions [3]. This component is designed as an aid to early visual detection.

The other components – land use analysis, WCLWD severity analysis and community advisory function – do not fall into the scope of this thesis on a single level. The genetic validation, prediction and planting suitability assessment were no further into the scope of this thesis.

1.7.2 Limitations

The number of images used is quite small and are imbalanced, with 118 Dwarf coconut images, 67 King Coconut images, and 243 Tall coconut images. The pictures gathered may not provide a broad visual spectrum, as they were limited to a few plantations, growth stages, backgrounds and conditions of capture.

We use the validation subset for checkpoint selection and episodic evaluation on the test set, so the results below are unlikely to be an estimate of test set performance. The episodes for evaluation can also be based on (reused) images in the validation set, so there are more query predictions than validation images.

The same three categories appear in both the training and validation data, and the results do not show generalization to coconut categories not seen before. The implemented classifier categorizes inputs into one of the existing categories, and there is no verified method for rejecting another category.

Grad-CAM heatmaps provide visual information that is associated with the models outputs, but they are not proof of genetic identity, nor do they prove that the region being focused on has biological significance. Visual appearance in isolation is unlikely to be enough for an expert to determine the quality of a saliency explanation. The visualization could be further explored to examine biological relevance.

Extents of performance with different locations and capture conditions have not been confirmed. According to some previous work in plant image classification, there is the potential for reduced performance when the images used in testing are not captured under the same conditions as those used for training. Independent field tests will be required for wider applications.

1.7.3 Assumptions

Assumptions Category Labels: The study assumes the correctness and consistency of the provided category labels which act as reference categories in model development and testing. It does not verify the labels via genetic tests of the images submitted. It is also assumed that images submitted show enough recognizable coconut detail to distinguish categories.

Objects with an excessive level of blur, objects occluded by another object, images with poor lighting, or unrelated images are likely not to conform with this assumption.

Input Images: It is assumed the input image is resized to an allowable size (usually larger, to be scaled down), and normalized as was the case during model development. The accuracy of the model also assumes that the stored class prototypes and class labels are consistent with the image label when incorporated. The prediction is to be used for initial identification support, with further confirmation from experts if plant status or agronomic appropriateness were to be determined.

1.8 Individual contribution

The individual contributions embodied in the thesis include design and assessment of the coconut image classification module in the integrated project context. This work primarily aims at differentiating between Dwarf, King Coconut and Tall coconut images and to visualize explanations that allow the inspection of the predictions.

The submission includes a dataset of 428 images for model training, which includes sorting of images, preprocessing, data augmentation, and splitting images by classes in training-validation. These will maintain a uniform input pipeline for both training and inference.

For model implementation we used an Image Net pretrained Efficient Net-B0 backbone with an embedding layer of size 512 and we used prototype classifier. We used episodic training, in which we build class prototypes from support embeddings and use cosine similarity as a score, along with validation-based checkpointing. Our contribution lay in the adaptation and application of these techniques to the selected coconut classes.

The assessment includes metrics: episodic classification accuracy; class-wise precision, recall and F1-score; the construction of a confusion matrix. We also provide the analysis of various categories' performance differences and indicate the limitations of the database, which include the reutilization of the validation set, the repetitive sampling of images and the absence of a test set.

The factor for explainability is to apply the Grad-CAM in the representative for the chosen similarity score, which produces the heatmap overlay of the input image. The

heatmaps can be qualitatively explored for the areas that are relevant for the predictions, such as the background.

In addition, the project accounts for the model weight creation, class prototypes, label categories and preprocessing for inference as well as web application use of the project. Overall, the project accounts for the implementation of a classification component, as well as the documentation of an evaluation method for early coconut identification. It is important to clarify that these contributions are not in the development of a learning algorithm nor the confirmation of genetics.

1.9 Thesis structure

This thesis is organized into nine chapters.

Chapter 1 introduces the overall project and the individual coconut image classification component. It presents the problem statement, research gap, aim, objectives, research questions, scope, limitations, assumptions, and individual contribution.

Chapter 2 reviews literature relevant to coconut identification, plant image classification, transfer learning, few-shot learning, and visual explanations. It compares previous studies and explains the foundation for the selected approach.

Chapter 3 describes the research methodology, including dataset preparation, preprocessing, augmentation, model development, episodic training, and evaluation procedures.

Chapter 4 presents the component's design, including its architecture, classification pipeline, prototype construction, visual explanation process, and interfaces with the overall application.

Chapter 5 explains implementation, integration, and testing. It describes the model inference process, Grad-CAM generation, and the preparation of the component for use within the web application.

Chapter 6 presents the results, including training and validation observations, episodic accuracy, class-specific metrics, the confusion matrix, and selected visual explanations.

Chapter 7 discusses the findings in relation to the research questions and previous literature. It examines classification errors, the interpretation of heatmaps, and the limitations of the evaluation.

Chapter 8 considers commercialization and sustainability, including potential users, deployment requirements, maintenance, and further validation needed for practical use.

Chapter 9 concludes the thesis by summarizing the findings and contributions and identifying directions for future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This chapter presents a summary of literature involved with the image-based coconut identification component. It considers coconut morphological classification, automated plant recognition, pretrained feature extraction, prototype-based learning and visual explanation techniques. The review provides a basis for the conclusions the selected methods and separates conclusions from earlier research from the evidence found during this research.

The study of coconut forms in Sri Lanka shows the importance of the morphological traits in the classification of coconut [4]. Closely related to image-based plant recognition studies are computational approaches for plant identification based on leaf images [5] and herbarium plant identification [6] using CNNs. These studies are linked to the learning of visual distinctions, but carry tasks and datasets different to the three categories of coconuts explored in this thesis.

The methodological review delves into EfficientNet as a feature extraction architecture [1] and Prototypical Networks as a classification with support examples and class representations approach [2]. In their case study, researchers are interested in applying them in the context of the small and unbalanced coconut dataset. The review also separates known categories being classified episodically from the evaluation of new categories if they are unknown.

The classification performance is viewed in conjunction with explainability and evaluation. They can also be used to identify regions in the image that are more important to the model's output, as found in the work of Grad-CAM [3], and understanding beyond the visual can be evaluated in the research on saliency evaluation [8]. Evidence of reduced plant classification performance under different image capture conditions further motivates careful interpretation of dataset-specific results [7]. These topics are explored in detail in the following sections and in relation to the design, evaluation, and limitations of the coconut classification section.

2.2 Domain and theoretical foundations

Germplasm Identification of the Coconut (Cocos nucifera L.) is the process of distinguishing the varieties and forms of the coconut based on the visible features. Sri Lanka has three main varieties of classification as described in the literature that includes Typica, Nana, Aurantiaca. The Tall coconut is represented by Typica, whilst the Dwarf coconut is represented by Nana and the intermediate types by Aurantiaca. Other morphological features are also taken into account in this classification apart from stature and breeding behavior [4, 9].

The forms of coconuts collected in germplasm exploration in Southern Sri Lanka were explored by Ekanayake et al. [4]. In addition to the last two items, they also evaluated stem characteristics, root bole formation, crown size, nut morphology and bearing patterns. Paper presented observations to support classification of some coconut forms that had not been represented. The authors were unable to verify some forms, however, due to a lack of samples. Their results underscore the need to consider several characteristics as well as to collect enough evidence in the selection of coconut germplasm.

Kamara et al. [9] provided additional evidence of the variation in coconut populations. The scientists analyzed 200 palms of Sri Lanka Yellow Dwarf population and looked at characteristics of stem, leaf, inflorescence, fruit and yield, as well as breeding behavior of the palms. They identified Sri Lanka Yellow Dwarf as a new variety, Sri Lanka Yellow Semi Tall, which they recognized as part of Aurantiaca. This discovery shows that palms with similar characteristics (in this case the color of the fruits are yellow) may need more observations to determine their classification.

A key implication of these studies is that the information within a photographic image may only reflect a small subset of the information used to determine the morphology in the traditional way. The image can be either captured with the colour and shape of fruit, but without full Palm height or growth shape or reproductive behaviour. As such, classification based on one image should be used as a preliminary visual assessment and not a full evaluation of the germplasm.

For the present research, the set of data will be categorized into the following three data sets: dwarf, king coconut, and tall. The labels represent the classification task and not all of the coconut forms described in the literature. The model is then tested to determine its performance for discriminating images designated to these selected classes. It cannot stand alone to establish the genetic identity, to resolve hybrid ancestry or to replace the assessment with other morphological or breeding information.

2.3 Related Research on Deep Learning for Plant Image Classification

Photographs of leaves, herbarium images and natural environment images have been used as inputs for identification of plants using deep learning. These applications are used as a basis to explore image based coconut classification. Their findings should be interpreted in the context of the specific data and identification problems in each study, however. To identify plant species from digitised herbarium specimens, Carranza-Rojas et al. [6] explored deep learning. They also studied relationships between herbarium images and field photographs, including the transfer learning of the latter between collections. The result showed that deep learning is a promising method to help in the identification of plants. However, the effectiveness of a classification task is not demonstrated with dried specimens mounted on herbarium sheets, because they are different from images of coconuts in the field.

A 26-layer residual network was used by Sun et al. [10] for identifying plants in natural environments. They reported a test accuracy of 91.78% for their model. The present study demonstrates the ability of deep learning to classify plant types in natural images. The reported accuracy, however, is for its own data set and evaluation procedure, and should not be directly compared to the episodic accuracy of the present coconut classifier.

Interestingly, Mohanty et al. [7] have used deep learning models to detect plant disease using leaf images, a similar job in the field of agriculture. While they were able to produce high accuracy on held-out images from the development set, they failed to perform well on images acquired under varying conditions. The result also points to the need to test models during use conditions other than those in which they were

developed. While the goals of disease recognition and germplasm identification are different, both need to be paid attention to so that image backgrounds, lighting, and acquisition conditions are considered.

Table 2.1 summarizes these studies and their relevance to the present research.

Table 2.1Comparison of Selected Deep Learning Studies for Plant Image Analysis

Study	**Task and image source**	**Main approach**	**Relevance to the present research**	**Limitation when applying its findings**
Carranza-Rojas *et al.* [6]	Species identification from digitized herbarium specimens	Deep learning and transfer learning across image collections	Supports the use of learned image representations for botanical identification	Herbarium specimens differ from field coconut photographs
Mohanty *et al.* [7]	Plant disease recognition from leaf images	Deep convolutional neural networks	Demonstrates the importance of testing under different image conditions	Disease categories differ from coconut germplasm categories
Sun *et al.* [10]	Plant identification from natural-environment photographs	A 26-layer residual network	Provides evidence for classification using natural plant images	Its dataset and evaluation procedure differ from the present study

Altogether, these papers indicate that exploring deep learning for visual plant classification is a direction worth pursuing but showing that the results are potentially task-, dataset-, and evaluation condition-dependent. In our case, the most significant difficulty seems to be discriminating between Dwarf, King Coconut, and Tall types of coconut given the constrained and unbalanced dataset, which is the reason to explore pretrained feature extraction and prototype based classification. Still, validation on the

provided imagery cannot directly translate to performance for different locations or unseen types of coconut.

2.4 Existing systems, products and techniques

Existing plant identification applications and machine learning techniques provide useful foundations for the proposed coconut germplasm identification component. This section examines their functions, relevance, and limitations in relation to classifying Dwarf, King Coconut, and Tall images from a limited dataset.

2.4.1 Existing plant identification systems and products

Pl@ntNet is an AI-powered citizen science platform for plant species identification. It has a web and mobile application for citizens to upload photos and get a prioritised list of species. Users can make comparisons with existing observations and share observations with the community [12]. These functions illustrate how image identification can be provided via an accessible application. But the documents reviewed do not provide validated accuracy of distinguishing the 3 categories of coconut that were selected for this research.

Find identification for plants and other organisms using Seek by iNaturalist. It's recognition technology relies on observations submitted to iNaturalist and partner sites, and recognized by the iNaturalist community [13]. It is an example of how computer vision could be used to assist with identification using a mobile application. However, the general identification of organisms does not correspond to the distinction of classes under coconut and the documented capabilities do not guarantee performance for the present class distinction task.

These products present practical cases of image submission and identification workflows to give the reader relevant examples. They are relevant to the proposed component mainly to the extent that they illustrate how users can be made to be able to be identified visually. The coconut part needs to be labeled data, evaluation process, and explicit restrictions of prediction.

2.4.2 Transfer learning and EfficientNet-based feature extraction

Transfer learning reuses representations learned from a source task when developing a model for a target task. Yosinski et al. [11] studied the transferability of features in

CNN and found that the features learned in the earlier layers are more general and the later layers are more specific to the source task. They also found that the more different the two tasks are, the less transferable they are. These observations align with the notion of using pre-trained representations and evaluating their appropriateness for the application.

Tan and Le [1] proposed Efficient Net using a compound scaling technique that considers width, depth and input resolution of a network. They showed good accuracy and computational cost on the tested benchmarks, which also included transfer learning datasets. This work lays the groundwork for evaluating EfficientNet as a feature extractor, but not the performance on coconut germplasm images.

In the current research, we are using ImageNet-pretrained EfficientNet-B0 as the backbone. Features are extracted and then compiled into a 512-dimensional embedding space for prototype-based classification. In the case of the coconut images, the available images are classified with pretraining, but it is not guaranteed to be classified successfully. It would take another controlled comparison with uniform data and evaluation processes to establish superiority over other backbones.

2.4.3 Few-shot learning and prototype-based classification

Prototypical Networks for recognising previously unseen classes using a small number of labelled examples were introduced by Snell et al. [2]. The method learns an embedding space and computes the average of the embeddings of the support examples to represent each class. Query examples are compared to these class prototypes to classify them. This gives a relatively simple way to classify tasks with relatively small amount of support data.

In the present research, the mechanism used is modified to compare query embedding with class prototypes by applying scaled cosine similarity. It is trained in an episodic way with three classes, eight support images per class and five query images per class. These choices are more related to the implementation that was adopted for this study, and not a new classification algorithm.

Similarly, the evaluation environment should also be separate from the original unseen-class objective. Dwarf, King Coconut and Tall are used in the present study for

both the training and the validation. For the reasons stated it is evaluated by means of new images of known categories. Not showing recognition of categories of coconut which were not trained.

2.4.4 Visual explanation techniques

Grad-CAM, proposed by Selvaraju et al. [3] is an approach that utilizes gradients of the desired model output to produce a heatmap of regions that contribute to the output. The classification score selected for implementation in the proposed component is based on a prototype (deep) learning approach and is complemented by Grad-CAM for visualizing predictions.

Explanation outputs need to be interpreted. Adebayo et al. [8] showed that visual evaluation can be misleading in assessing saliency methods and suggested some tests to check the dependency of the methods on model parameters and training data. The results reveal why an appealing heatmap is no substitute for a regular explanation.

Therefore, qualitative inspection was made in the current research by using Grad-CAM. The heatmaps can be used to analyze if the regions highlighted in them intersect with visible coconut structures or background areas. They are not able to independently determine the correctness of classification, relevance of biology or genetic identity. A comparison of the reviewed products and techniques, and a summary of their relevance to the proposed component are provided in table 2.2.

Table 2.2Comparison of Existing Products and Techniques Relevant to Coconut Image Classification

Product or technique	**Main function**	**Relevance to the proposed component**	**Limitation or evaluation requirement**
Pl@ntNet [12]	Plant species identification from photographs	Provides an example of image submission and identification workflows	Reviewed documentation does not establish accuracy for the selected coconut categories

Seek by iNaturalist [13]	Visual identification of plants and other organisms	Demonstrates accessible identification through a mobile application	Broad organism identification differs from coconut category classification
Transfer learning [11]	Reuses representations learned on a source task	Provides an initial representation for learning from limited images	Effectiveness depends on the relationship between source and target tasks
EfficientNet [1]	Generates convolutional image features	Provides the pretrained backbone used in this study	Published benchmark results require verification on the coconut dataset
Prototypical Networks [2]	Classifies query examples using class prototypes	Supports classification from small support sets	Known-class evaluation does not establish unseen-class recognition
Grad-CAM [3]	Highlights regions associated with a selected model output	Supports qualitative inspection of predictions	Heatmaps require careful interpretation and further validation [8]

The reviewed products and techniques support investigating a dedicated coconut identification component that combines accessible image submission, pretrained feature extraction, prototype-based classification, and visual inspection. The contribution of this research lies in implementing and evaluating this combination for the selected coconut categories within the integrated coconut management platform.

2.5 Gap synthesis and conceptual framework

Foundations for morphological identification, image-based classification of plants, transfer learning, prototype-based learning, and visual explanations were found in the

reviewed literature. These foundations cannot, however, claim to be a proof of the effectiveness of an integrated classifier for the selected coconut categories. The gap in the research that this study is tackling is thus application-specific: creating and assessing a component that is able to classify Dwarf, King Coconut, and Tall images from a small number of labelled images, with the ability to view and inspect the results of the component.

2.5.1 Gap synthesis

Morphological study shows that the coconut classification involves several characteristics such as stature, coconut breeding behavior and stem, crown and fruit features [4], [9]. Some or all of these may not be represented in a photograph. Hence, an image-classifier can be used to provide initial identification, but can never be used as a full morphological assessment or genetic verification.

Previous studies on plant image classification shows the applicability of deep learning on various types of image sources [5, 6, 10]. However, accuracy of those datasets is not reflected in the coconut categories used here. Reduced performance under varying image conditions also demonstrates the importance of making a distinction between validation in a development dataset and generalization to other field conditions [7]. EfficientNet and transfer learning lay the foundations for adaptation of pre-trained visual representations [1, 11] and Prototypical Networks offer a way to classify with small support sets [2]. The combination of these two is a reasonable way to explore them with the data available on coconut. The effectiveness, however, has to be tested and not assumed, and categories have to be distinguished from which have been evaluated before from those not.

Grad-CAM is an approach to examine areas related to a chosen model output [3]. These inspection methods are used in conjunction with classification metrics, and cannot be used to show that predictions depend on biologically meaningful features. The constraints that were outlined in the saliency research are thus relevant to the interpretation of visual explanations [8].

Table 2.3 summarizes these findings and the corresponding responses adopted in this research.

Table 2.3Synthesis of Literature Findings and Research Responses

Literature finding	**Application-specific issue**	**Response in this research**
Coconut identification considers multiple morphological and breeding characteristics [4], [9]	A single photograph may contain incomplete identification evidence	Define the output as preliminary visual classification of three selected categories
Deep learning supports plant image identification [5], [6], [10]	Published results do not establish performance on the available coconut dataset	Evaluate the classifier using the collected coconut images
Performance can decrease under different image conditions [7]	Validation results may not represent other locations or acquisition conditions	State the limits of the evaluation and identify independent field testing as future work
Pretrained features can transfer between tasks, with varying effectiveness [1], [11]	The suitability of pretrained representations for coconut images is uncertain	Adapt EfficientNet-B0 and evaluate the resulting classification component
Prototype-based learning supports classification from small support sets [2]	Limited data require an approach whose effectiveness can be investigated experimentally	Use support-based class prototypes and episodic training
Visual explanations require careful validation [3], [8]	A heatmap may appear convincing without confirming correctness	Use Grad-CAM for qualitative inspection and avoid treating it as biological proof

2.5.2 Conceptual framework

Conceptual Framework: relates the available image data, learned representations, classification mechanism, and evaluation evidence. The inputs to the development and evaluation of the component is the labelled coconut image. The image preprocessing defines a consistent format of the input image and the training augmentation creates variation in the training image.

Each image is passed to an ImageNet-pretrained EfficientNet-B0 backbone, which extracts features from it. The following features are embedded in 512 dimensions.The following features are projected into a 512 dimensional embedding. Within each category, the embeddings are averaged to create class prototypes; Then, query embeddings are compared to these prototypes using scaled cosine similarity. The predicted label is the one that has the highest comparison score.

The grad-CAM gives a complementary visual output by pinpointing image regions corresponding to image classification scores. The accuracy, precision, recall, F1-score and confusion matrix are used to analyze the classification performance. Visual explanations are examined qualitatively to look at the highlighted regions of the correct and incorrect examples.

The framework decomposes the evidence into two categories: quantitative evidence of classification performance and qualitative evidence of the highlighted image regions of the model. Neither is considered to establish genetic identity. Moreover, training and validation are the same three categories, and thus the evaluation is not on unseen-category recognition; it is only on known-category classification.

The outputs generated are prediction and a visual explanation that are to be presented through the integrated web platform. The framework does not take for granted that using prototype-based learning is more effective than using other classifiers, nor does it take for granted that using Grad-CAM is more effective than using the other methods.

3 RESEARCH METHODOLOGY

3.1 Research design

The study integrates an Artificial Intelligence-based component for classification of coconut germplasm with an experimental test of its performance. The development process is organized according to the proposed design science research methodology by Peffers et al. [14] comprising the problem identification and motivation, definition of solution objectives, design and development, demonstration, evaluation, and communication. The structure of the development activities is given by the framework and the classification performance in the provided dataset is evaluated by the experiments.

This relationship of the individual component and the stages is shown in figure 3.1.

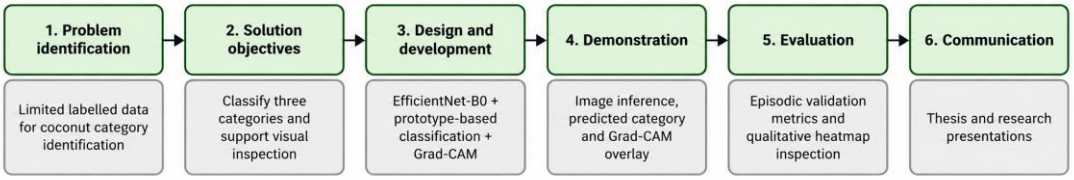


Figure 3.1:Development and evaluation workflow for the coconut germplasm classification component, adapted from [14].

The coconut germplasm classification component development and evaluation process are adapted from [14] and are shown in Figure 3.1.

The problem identification stage is used to illustrate the need to classify the Dwarf, King Coconut, and Tall images in a preliminary classification, limited labelled data are required. The research does not deal with all morphological identification or genetic verification, but rather this particular classification task. The purpose of the solution is to forecast one of the three chosen categories, to present the product's predictions graphically to be compared, and to assemble the component for integration into a product.

In the design and development phase, the visual features are extracted by an ImageNet-pretrained EfficientNet-B0 backbone, and then projected into a 512-dimensional embedding [1]. The classification is done by using a class-prototype principle as is done in Prototypical Networks [2]. The support images are averaged within each category to form prototypes, and the query image is matched to the prototypes via cosine similarity scaled by a learnable value. Grad-CAM is integrated to produce visualisations related to a chosen classification score [3]. These options are all adaptations of the existing techniques that are application-specific.

The procedure employed in the experiment is a class-wise split of 428 images of labelled coconut, comprising of 341 training images and 87 validation images. The training is carried out in an episodic experimental setup with 3-way, 8 shots, and 5 queries. Each episode contains all three categories, 8 support images and 5 query images for each category. Support images create class prototypes; query images are used to compute the training loss or validation metrics.

In the demonstration stage, the trained component is used to process the input images and it generates a category prediction and a Grad-CAM overlay. The inference artefacts prepared to be integrated with the web platform include the encoder, fixed class prototypes, class labels and the pre-processing configuration. It is important to note that this workflow demonstrates its functionality, but doesn't necessarily prove its reliability under various field conditions.

The evaluation stage consists of a mixed method of quantitative classification measures and qualitative visual inspection. Classification performance and differences between classification categories are described by Accuracy, Precision, Recall, F1 score and a confusion matrix. Grad-CAM heat maps enable the examination of areas linked to the chosen predictions, such as if the highlighted areas are overlapping with the coconut structures or with the background. But visual plausibility is not sufficient to ensure reliability of explanation, correctness of prediction, or biological relevance [8].

The three categories are present in both the training set and the validation set. Thus, the evaluation test is not to recognize new category but to classify new images into

known categories. The results also provide a reference for validation performance, and are therefore not an independent test of the final validation. The number of repeated query predictions over validation episodes is different than number of unique validation images.

Episodic validation and application inference are also distinguished. Validation prototypes are built from the support images sampled during each validation episode while application inference uses fixed prototypes based on the training images. So it is not considered as a direct measure of application performance when deployed.

Lastly, the communication stage introduces the methods, findings, individual contribution and limitations in the thesis and research presentations. The development workflow and experimental procedure offer a framework for evaluating the component without claiming to be superior to other classifiers or more effective than the one assessed in the experiment.

3.2 Data participants or study materials

The main investigation material is a set of labelled images. The task to be performed in the experiment is to classify the image of the coconut into three given classes: Dwarf, King Coconut, and Tall. The model-training and validation experiments, discussed above, do not include human participants.

3.2.1 Dataset source and composition

The project dataset comprises 428 images of coconuts, captured from the Coconut Research Institute premises at Bandirippuwa with the aid of a Nikon D5600 camera, along with the images captured using a Samsung A16 smart phone. The images are grouped in category folders, with the labels being the category names used in the training notebook. These labels represent the experimental classification task and do not convey any validation information.

The dataset includes 118 Dwarf images, 67 King Coconut images and 243 Tall images. The category distribution, as well as the training–validation partition, recorded in the notebook, is shown in Tab. 3.1.

Table 3.1:Coconut image dataset composition and training–validation split

Category	Total images	Training images	Validation images
Dwarf	118	94	24
King Coconut	67	53	14
Tall	243	194	49
Total	**428**	**341**	**87**

This is not even distribution as Tall is the largest and King Coconut is the smallest. Episodic sampling randomly chooses the same number of support and query images from each category in an episode. This helps to keep categories in individual episodes and will not add more or different images.

3.2.2 Training and validation partition

This data is split in each category with about 80% being training images and 20% validation images. The resulting pools are 341 and 87 images in size, respectively. The notebook is set to repeat the random operations by setting a random seed with the value 42.

Training pool is used to optimize model. The validation pool is employed to evaluate performance while developing the model and to choose the checkpoint of the model. The Dwarf, King Coconut and Tall images are present in both the pools. This partition enables the evaluation of novel images that belong to a set of known categories, but not a set of categories not used for training.

The partition is at image level. If no record is provided that connects a picture to a particular palm, it can say nothing about whether the training and validation sets are completely different palms. Several pictures of the same palm or of similar conditions might then lessen independence of the evaluation. This is kept as a limitation for further research where a partition at the palm-level or the location-level is needed.

3.2.3 Maintain and seek information for study

The sample of training and validation episodes come from their respective image pools. The 8 support images and 5 query images for each episode equals a total of 24 support images and 15 query images for the three episodes.

Images are embedded and used to build class prototypes. The training loss or the evaluation measures are determined by comparing query images with these prototypes. Samples might be replayed in later episodes. As a result, the total number of predictions of queries does not mean the same number of images or independent observations.

Fixed prototypes are inferred from 20 training images, categorized by selected checkpoint for application inference. These prototypes are independent of the prototype building from validation support images in episodic evaluation. It is maintained in the interpretation of the model results and description of application behavior.

3.2.4 Data coverage and limitations

The dataset offers a variety of material for exploring classification in the three chosen categories. Within this class its size and imbalance, plus its collection environment and perhaps associated pictures, restrict the conclusions that can be drawn about field performance in general. It is not a representative sample of coconut forms, growth stages, location or photographic conditions.

In the study, predictions are therefore considered as initial categorizations of the image. The scope of evaluation does not include genetic identity, reliable recognition of unknown categories and suitability for plantation. Additional assessment will depend on photos collected by the researchers, verified tags, and parts that will incorporate individual palms and collection sites.

3.3 Experimental or study procedure

The experimental process includes image preprocessing, episodic sampling, model training, selecting checkpoints, validation and preparing inference artefacts. Qualitative inspection of selected predictions is done with the help of Grad-CAM visualizations. The procedure is carried out as described in the research notebook.

3.3.1 Image preprocessing and augmentation.

The images are uploaded from their respective categories and then resized to 224 × 224 pixels, converted to RGB and also centered. A smaller size is used for the images, which are then stored in memory for faster access when sampling periodically. Images are converted into tensors and normalized using channel means of [0.485, 0.456, 0.406] and standard deviations of [0.229, 0.224, 0.225].

Training Augmentation includes random horizontal flipping, random rotation within +-20° and color jitter with brightness, contrast and saturation of 0.3. These changes help to add controlled variation but will not lead to an increase in the number of images collected independently. Images are augmented during training for both the support and query sets. Validation and inference are done in the absence of random augmentations and with resizing and normalization.

3.3.2 Episodic sampling

There is a Dwarf, King Coconut and Tall in each episode. The number of images sampled from each category are 8 images (support set) and 5 images (query set). The number of support images and query images in each episode are thus 24 and 15, respectively.

For the data that is available, images are sampled without replacement, for each category, during an episode. This won't cause a picture to be seen in both sets of the support set and query set of any particular episode. Some images might be shown again in later episodes.

The trainings are sampled from the training pool and the validations are sampled from the validation pool. The number of samples taken from each category is balanced within each episode even though the distribution of the overall data set may not be equal across categories.

3.3.3 Feature extraction and prototype-based classification

This introduces the concept of feature extraction and prototype-based classification. The embedding model is based on the convolutional architecture of an ImageNet pre-trained EfficientNet-B0 backbone [1]. The feature maps are then fed into an adaptive

average pooling layer, which outputs a 1,280-dimensional vector. The embedding is a 512-dimensional projection, followed by a batch normalization.

Following the class-prototype principle of Prototypical Networks [2], each category is represented by the mean embedding of its support images:

$$\mathbf{c}_k = \frac{1}{|S_k|} \sum_{\mathbf{x}_i \in S_k} f_\theta\,(\mathbf{x}_i). \tag{3.1}$$

Here, $S_k$is the support set for category (k), $(|S_k|)$is its number of images, $f_\theta$is the embedding model, and $\mathbf{c}_k$ is the resulting prototype. During episodic training and validation, $(|S_k| = 8)$.

A query image is compared with each prototype using scaled cosine similarity:

$$z_k(\mathbf{x}) = s\,\frac{f_\theta(\mathbf{x})^\top \mathbf{c}_k}{\|f_\theta(\mathbf{x})\|_2 \|\mathbf{c}_k\|_2}. \tag{3.2}$$

In Equation (3.2), $z_k(\mathbf{x})$is the classification score for category k, and s is a learnable scale parameter initialised to 10. The implementation normalises the query embeddings and prototypes before calculating their dot products. Although the code names the parameter log_scale, it is used directly as a multiplier rather than exponentiated.

SoftMax converts the classification scores into normalized class outputs:

$$p_\theta(y = k \mid \mathbf{x}) = \frac{\exp\left(z_k(\mathbf{x})\right)}{\sum_{j=1}^{K} \exp\left(z_j(\mathbf{x})\right)}. \tag{3.3}$$

Here, $K = 3$is the number of categories. The predicted category is the one with the highest classification score, which also has the highest softmax output. The probability notation in Equation (3.3) describes the model's categorical distribution; it does not establish that these outputs are calibrated probabilities of prediction correctness.

3.3.4 Model training and checkpoint selection

The model is trained for 40 epochs, where 40 training episodes are given per epoch. The labels and query classification scores are fed into cross-entropy loss. The query embeddings are propagated through the gradients to update the embedding model and scale parameter, with the support-derived prototypes.

Adam is used for optimization, with an initial learning rate of 1×10^{-4} .A 40 epoch learning rate scheduler that performs cosine annealing is configured. The EfficientNet-B0 backbone is optimized along with the projection layer and is not used as a pre-trained feature extractor.

The model is assessed after every epoch for 15 epochs of validation without augmentation or parameter update. We select the checkpoints based on mean query accuracy. If this accuracy is higher than the previous Saviours accuracy, it is saved as is a Checkpoint.

Table 3.2 summarizes the principal experimental settings.

Table 3.2:Experimental configuration for episodic model training and validation

Setting	**Value**
Input image size	224 × 224 pixels
Backbone	ImageNet-pretrained EfficientNet-B0
Embedding dimension	512
Categories per episode	3
Support images per category	8
Query images per category	5
Training epochs	40
Training episodes per epoch	40
Validation episodes per epoch	15
Optimiser	Adam

Initial learning rate	1×10^{-4}
Learning-rate scheduler	Cosine annealing
Training loss	Cross-entropy
Similarity measure	Scaled cosine similarity
Scale initialization	10
Checkpoint selection	Highest mean episodic validation accuracy
Post-training evaluation episodes	50

Figure 3.2 illustrates the support and query processing paths, prototype construction, and loss calculation used during episodic training, together with the fixed-prototype inference procedure.

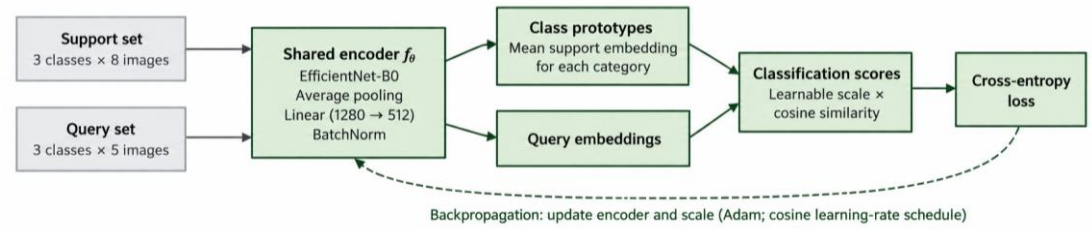


Figure 3.2: Episodic training and fixed-prototype inference procedure for the coconut classification component.

3.3.5 Post-training episodic evaluation

The checkpoint that was selected is reloaded and tested on 50 more episodes from the same validation set. The total number of query predictions is 750 (15 query images per episode). There are 250 predictions per episode for each category. Labels are predicted and actual labels are availed to create the classification report and confusion matrix. These predictions are based on repeated sampling of the 87 validation images and are NOT 750 test images. Since the validation pool can also be used for checkpoint, this method is not considered independent testing but rather post-training validation.

3.3.6 Fixed-prototype inference and visual inspection

Fixed prototypes are created from the first 20 images in the cache of training images for each category, for individual-image inference. In evaluation mode, their

embeddings are calculated with the selected checkpoint and averaged by category following Equation (3.1), where ($|S_k| = 20$).

The size of an input image is resized and normalized prior to embedding it with the fixed prototypes to be compared with using Equation (3.2). The winning score is used to predict the category and Equation (3.3) calculates the normalized class outputs. This inference procedure is not episodic inference, which would create prototypes from validation support images.

Grad-CAM is based on the last convolutional layer of the embedding model [3]. The selected category's scaled cosine-similarity score is then backpropagated to find gradients for the feature maps. The feature maps are weighted by spatially averaged gradients and the weighted combination of the feature maps is passed through a rectified linear operation and normalized to create a heatmap.

Resize and make a blend of the heatmap with the input image for visual inspection. These visualizations facilitate the review of areas related to classification scores selected but are not a measure of the accuracy of the prediction, its biological significance or genetic identity.

3.3.7 Export of inference artefacts

The notebook saves exportable model weights, weights for the embedding model in TorchScript and ONNX, fixed class prototypes, category labels, and preprocessing parameters. The exported embedding model generates feature embeddings and the prototypes and similarity calculation are used to make predictions of categories.

A manual inference example is used to check the exported prediction workflow. This functional demonstration is different to a more comprehensive evaluation of the whole application, which would take place through further testing.

Use the Word equation editor to enter Equations (3.1) through (3.3) in the center of each line, numbered to the right. The equations are numbered independently of the figures. None new references are required in this section, references [1], [2] and [3] are reused.

3.4 Measures and evaluation metrics

Let $(TP_{k)}$, $(FP_{k)}$,and FN_kdenote the true positives, false positives, and false negatives for category k, respectively. Classification performance was evaluated using the following measures:

- **Accuracy:** The proportion of evaluated query predictions that match their ground-truth labels.
- **Precision:** For category (k), $\frac{(TP_k}{(TP_k+FP_k))}$, representing the proportion of predictions assigned to that category that are correct.
- **Recall:** For category (k), $\frac{(TP_k}{(TP_k+FN_k))}$representing the proportion of ground-truth examples of that category correctly identified.
- **F1-score:** The harmonic mean of precision and recall, calculated as $\frac{(2PR}{(P+R))}$,where P and R denote precision and recall.
- **Macro-averaged precision, recall, and F1-score:** The unweighted mean of each measure across the three categories, giving each category equal importance.
- **Weighted-averaged precision, recall, and F1-score:** The mean weighted by each category's ground-truth query count. Each category contributes 250 query predictions in the post-training evaluation, so these averages equal the corresponding macro averages.
- **Confusion matrix:** A summary of ground-truth and predicted categories showing the frequency and direction of classification errors.

The training loss, training-episode accuracy, and validation-episode accuracy were measured and reported for each epoch to assess convergence and potential overfitting. Checkpoints were selected based on Mean accuracy for 15 validation episodes. The chosen checkpoint was then tested for more than 50 episodes to generate 750 predictions for queries repeatedly sampled from the 87 validation images. These predictions thus are not made on 750 independent or different test images.

Qualitative inspection of Grad-CAM outputs was performed to check if highlighted regions overlapped with salient visible coconut structures (nuts, crown

and fronds) or background regions. This inspection was not part of a morphological relevance study validated by an expert. Heatmaps were used as a visual supplement to the evidence of the correctness of the prediction and not as a proof of biological reasoning [8].

The exported inference workflow was checked using a manual prediction example. This successfully loaded and predicted this demonstrated model but did not prove numerical equivalence across all export formats. Episodic validation results were also distinguished from application inference using fixed training-derived prototypes.

3.5 Data analysis

Data analysis is a combination of examining the distribution of the data set, the training behavior, the classification performance and the selected Grad-CAM outputs. The imbalance in the original data set and the makeup of the training and validation pools are provided by category counts. These counts are distinct from the sampling done within episodes that is performed in a balanced way.

The convergence, performance variations, and potential overfitting of the model are analyzed through plotting the three metrics—mean training loss, training query accuracy and validation query accuracy—over 40 epochs. The checkpoint showing the best mean validation accuracy is chosen for further testing.

The labels for 50 additional training episodes (after training) are gathered to create a classification report and confusion matrix. Overall performance is best captured with accuracy, differences between Dwarf, King Coconut, and Tall are captured by differences in per-category precision, recall and F1-score. Confusion matrix shows the number and pattern of classification mistakes. The 750 query predictions are not considered as independent test observations because they are repeated samples from 87 validation images. No statistics are stated or significance claims made, and no confidence intervals are included.

Overlays obtained from the selected Grad-CAM models are checked to determine if the highlighted areas correspond to the visible structures of the coconut or the unrelated background area [3]. No quantitative explanation quality test or annotation

by experts is available, so this analysis is of a qualitative type. The visualizations cannot stand alone as proof of the correctness of prediction or its biological relevance [8].

Manual inference examples show the model loading and prediction capabilities for prototypes prefixed in the training process. This does not mean they are used as episodic validation or to guarantee application-wide accuracy or numerical equivalence between export formats. Conclusions are based on the data and on what has been observed.

3.6 Ethics privacy security and responsible AI

The course also covers privacy, security and responsible AI. Privacy, security and responsible AI are also addressed in the course.

Coconut images are used to develop the model instead of the data from human participants. However, photographs may also contain information and/or identifiable people, property or location by chance. Images should thus be checked prior to publication or sharing and identifying information removed where possible. There are different permissions associated with collecting images and permissions for redistribution, and the notebook available does not address collection permissions, data set ownership, or rights for redistribution. These facts need to be verified prior to public release.

Uploads of applications also need to be handled appropriately. Images, user identities and location data should be captured only if required for the purpose for which they are being captured and should include information about the use and retention of these images. Restrictions on access, secure transfer, validation of uploads and deletion policies are relevant safeguards. But the model experiments do not constitute proof that the controls have actually been put in place and security-tested in the integrated application.

To report the component accurately, the component's range is limited to preliminary visual classification of Dwarf, King Coconut and Tall. The model does not confirm genetic identity, palatability or reliably exclude unknown categories. The limitations

should be communicated to users and they should be encouraged to seek specialist assessment when it is necessary to confirm the identification.

While Grad-CAM visualizations facilitate the inspection of regions linked to a selected prediction [3], they are not able to confirm biological relevance and the correctness of the prediction [8]. Likewise, the uncalibrated softmax scores should not be given as true probabilities of correctness. It is recognized when interpreting performance that the dataset imbalance, potential overlap of palms in the different image pools, and limited collection coverage are factors. Independent field evaluation and expert validation is required to enable wider use.

3.7 Reproducibility and research artefacts

The research notebook contains data organization, pre-processing details, model architecture, episodic configuration, training procedure, model evaluation results etc. Python, NumPy, and PyTorch random seeds are set to 42 to support repeatability. But, the same results cannot be guaranteed with fixed seeds in different hardware and software environments. The notebook supports cuDNN benchmarking and is not deterministic.

The exact same image files, category labels, training/validation partition, software environment, and sequence of the experiment are all required to replicate the experiment. Episodic sampling and augmentation involve random operations and the order of execution could change the episodes sampled. The image-level split and the images used for creating fixed prototypes should be retained for following up reported results.

Table 3.4 summarizes the principal research artefacts produced by the notebook.

Table 3.3: Research artefacts and their purposes

Research artefact	Purpose
Research notebook	Records preprocessing, training, evaluation, export, and Grad-CAM procedures
best_model.pth	Stores the selected model checkpoint and learned similarity scale

model_weights.pth	Stores the encoder weights, scale, and embedding dimension
model_final.pt	Provides a TorchScript version of the embedding model
model_final.onnx	Provides an ONNX version of the embedding model
prototypes.pt	Stores fixed prototypes derived from training images
label.json	Maps output indices to the three category names
preprocessing_config.json	Records image size, normalization values, scale, and similarity settings
Recorded evaluation outputs	Provide training curves, the classification report, and confusion matrix

The exported embedding models need the class prototypes and similarity calculation to make the predictions of the category. The processing set-up should also be consistent with the training process. A manual example is provided to show the exported inference workflow, but there is no systematic proof of numerical equivalence between the different export formats.

These artefacts help to trace back the experiment that was conducted. In addition to the above, a complete reproducibility package should include dependency versions, the dataset split, and prototype-source image identifiers (if data sharing is agreed upon).

4 4 PRODUCT AND SOLUTION DESIGN

4.1.1 Requirements

The components required are based on the individual research goals and the role of the research in the integral coconut management platform. The main purpose is to take as an input an image, and return a preliminary classification of Dwarf, King Coconut, or Tall, and a visualization of the Grad-CAM. The requirements defines the desired behavior of the component, but their implementation and verification are discussed in Chapter 5.

Table 4.1:Functional requirements of the coconut classification component

ID	Requirement
FR01	Accept an uploaded image for coconut category classification.
FR02	Validate that the uploaded file can be decoded as an image and return an appropriate error for invalid input.
FR03	Apply RGB conversion, resizing, and normalization consistent with the trained model.
FR04	Generate an embedding and compare it with fixed prototypes for Dwarf, King Coconut, and Tall.
FR05	Return the category with the highest scaled cosine-similarity score.
FR06	Generate and display a Grad-CAM overlay associated with the selected category score.
FR07	Communicate that the output is preliminary visual classification and does not establish genetic identity or planting suitability.
FR08	Provide a prediction interface that can be integrated with the common web platform.

Table 4.2 presents the non-functional requirements.

Table 4.2:Non-functional requirements of the coconut classification component

ID	Requirement
NFR01	**Consistency:** Use the selected checkpoint, matching prototypes, category mapping, and preprocessing configuration together.
NFR02	**Usability:** Present the predicted category and visualization clearly, with understandable error messages and limitations.
NFR03	**Reliability:** Handle invalid inputs and unavailable model artefacts without presenting a misleading result.
NFR04	**Performance:** Measure prediction and Grad-CAM response times in the target environment before establishing a latency target.
NFR05	**Privacy and security:** Restrict access to stored images and associated records, and apply appropriate upload safeguards.
NFR06	**Maintainability:** Keep preprocessing, embedding generation, prototype comparison, and visualization sufficiently modular to support updates.
NFR07	**Traceability:** Retain model and configuration versions so predictions can be associated with the inference artefacts used.

The classification scope is limited to the 3 categories selected. The current model does not have a reliable rejection of unrelated images or unknown coconut categories. The interface should therefore clarify what is accepted as input to the system and what types of images are classified, not suggest that all images uploaded can be accurately classified.

If providing prediction scores, they should be referred to as normalized model scores, not as probabilities of correctness. Likewise, grad-CAMs should be displayed as visualization tools for inspection. The security, response time and usability requirements are not covered by classification results, but must be verified on application level.

4.2 Component architecture within the complete solution

The germplasm classification module of coconut functions as an individual prediction service, part of the integrated coconut management platform. The platform integrates land use analysis, WCLWD severity analysis, germplasm identification and advisory support on a single web application. This individual module provides the initial category prediction along with the Grad-CAM visualisations.

This solution consists of presentation layer, application layer, inference layer and model artefacts. The presentation layer, based on React, provides the interface for image input and output, while the application layer based on Flask is responsible for receiving the requests, data preprocessing, classification and visualisation. These tasks are presented as the responsibilities of each component on Figure 4.1.

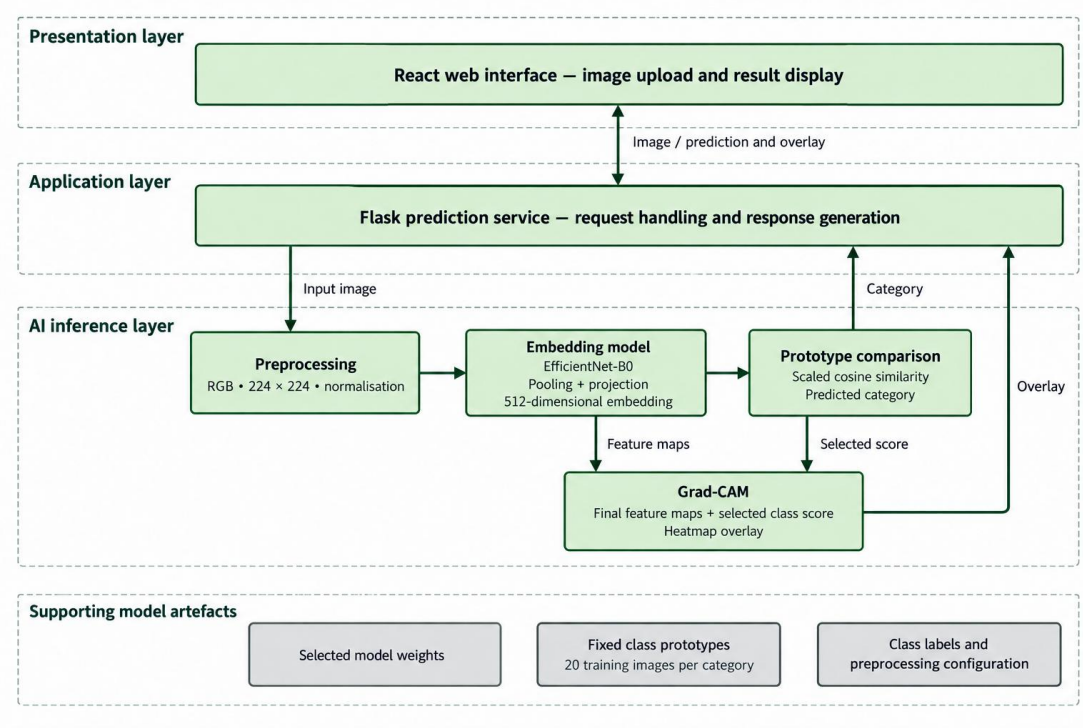


Figure 4.1 : Architecture of the coconut germplasm classification component within the integrated platform.

Once the image is uploaded, the inference pipeline turns it to RGB, rescales it to 224 × 224 px, normalises it with the same parameters as during training. The trained Efficient Net B0 backbone generates convolutional features that are pooled and projected to give a 512-dimensional embedding, which is compared against the

constant prototypes for the three categories Dwarf, King Coconut and Tall with scaled cosine similarity. The highest-scoring category is the prediction.

Grad-CAM builds a heatmap from the last convolutional feature maps and the similarity score of the selected category. The gradients of the score estimate the weights which combine the feature maps into the heatmap. The heatmap is resized and superimposed on the input image. The prediction service outputs the category and visualization to the interface.

Supporting artefacts shall be the chosen model weights, fixed prototypes (from 20 training images per class), the class labels, and preprocessing configuration. The artefacts shall be identical: the prototypes shall be for the chosen encoder, and the labels and preprocessing for the inference.

Table 4.3 summarizes the architectural elements.

Table 4.3: Architectural elements of the classification component

Element	**Responsibility**
React interface	Supports image submission and displays results
Flask prediction service	Coordinates requests, inference, and responses
Preprocessing module	Applies RGB conversion, resizing, and normalization
Embedding model	Produces a 512-dimensional image representation
Prototype comparison	Calculates category scores and selects the predicted label
Grad-CAM module	Produces a heatmap overlay associated with the selected score
Supporting model artefacts	Supplies model weights, prototypes, labels, and configuration

Integration happens at the prediction interface and the shared interface in the common platform. Sharing an interface does not mean that all four components use the same training data or that a germplasm prediction automatically drives another component.

Any movement of outputs necessitates an integration workflow. The structure separates application inference from episodic validation. Validation prototypes are made from validation support images, not training support. Application inference needs to be validated using a configuration with the fixed training support prototypes. This diagram only shows the working flow of classification; storage, authentication, and operational controls also require implementation evidence and testing.

4.3 Design alternatives and decisions

The component combines pretrained EfficientNet-B0, prototype-based classification, and Grad-CAM. Pretrained feature extraction provides an initial visual representation for fine-tuning with the limited coconut dataset [1], [11]. Prototype-based classification represents each category through support-image embeddings [2], while Grad-CAM supports visual inspection of selected predictions [3].

Table 4.4:Design alternatives and selected approaches

Design aspect	**Alternative**	**Selected approach**
Feature learning	CNN trained from scratch	Fine-tuned ImageNet-pretrained EfficientNet-B0
Classification	Conventional classification head	Episodic prototype-based classification
Similarity measure	Euclidean distance	Scaled cosine similarity
Inference prototypes	Support set supplied for each request	Fixed prototypes from 20 training images per category
Prediction display	Category label only	Category label and Grad-CAM overlay
Application structure	Combined interface and inference logic	React interface with a Flask prediction service

These choices support the intended workflow but have not been established as superior through controlled comparisons. Fixed-prototype quality may depend on the selected

images, and Grad-CAM does not independently confirm biological relevance or prediction correctness [8].

4.4 Detailed design

The component separates preprocessing, classification, and visualization into distinct modules. The prediction service coordinates these modules and returns results to the web interface.

4.4.1 Preprocessing and embedding generation

The preprocessing module takes in an image, converts it to RGB, resizes to 224 × 224 and applies normalization factors observed in the model configuration. The embedding module takes the convolutional features from EfficientNet-B0, applies adaptive average pooling and a linear projection with batch normalization to produce a 512-vector.

4.4.2 Prototype comparison and visualization

The classification module has three fixed prototypes: Dwarf, King Coconut and Tall. These are the mean embeddings of 20 training images using the selected checkpoint. Normalized image embeddings and prototypes are compared using scaled cosine similarity and the class with the highest score is used as the prediction, with label mapping from class index to text shown.

The Grad-CAM module takes the last convolutional feature maps, as well as the gradients of the chosen category score, and constructs a normalized heatmap. This heatmap is mixed with the input image. Gradients can be computed for visualization purposes, but model parameters are not affected.

4.4.3 Service coordination and artefact consistency

Flask prediction service integrates preprocessing, classification, and visualisation and sends back the class predicted, and overlay, to the React interface. Input must reject missing or abnormal files, such as not decodable ones, with equally clear reporting of fails to process. When an image is decodable it may still not be coconut. Rejection of such inputs is out of scope.

The following should be the same across experiments: model weights, prototypes, labels, and pre-processing. If changing the encoder, the prototypes should be re-created so that the embeddings of the queries and the class representations are generated from the same model.

4.5 Standards and professional practice

Consistent processing, traceability of model artefacts, modular implementation, and correct reporting are the focus of component design. The model stores image preprocessing and category mappings and the prototypes used to minimize inconsistencies between development and inference and to facilitate reproducibility. Separation of the processing, classification and visualization stages facilitates maintenance and testing.

Reported research makes a distinction between shown capabilities and suggested needs. The outcomes of episodic validation are labeled as evidence at the advancement stage, while the application's reliability, safety, and response time must be verified independently. IEEE citations offer information on the methods utilized in the paper, such as EfficientNet, Prototypical Networks, and Grad-CAM [1]–[3].

Privacy and security issues such as validating uploaded files, limiting access to stored records and determining suitable retention arrangements are additional design requirements, not insecurity evidence. The ability to collect and redistribute datasets needs to be clarified before any images can be distributed publicly.

Presentations should be in a professional format with category labels that are visually interpretable and simple error messages with an accessible explanation of the component's boundaries. Descriptions of predictions should indicate that they are qualified visual tags, and Grad-CAM overlay images should be printed as diagnostic tools, not biological evidence [8]. The software is not claimed to adhere to an established quality or security standard without a documented evaluation.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Model development was carried out in a Python notebook using the Kaggle environment, as indicated by the dataset and output paths in the supplied notebook. The implementation selects CUDA when available and otherwise uses the CPU. The application uses a React frontend and a Flask backend to connect image submission with the classification workflow.

Table 5.1 summarizes the principal technologies and their roles.

Table 5.1: Development environment and technology stack

Technology	Role
Python	Model development, preprocessing, inference, and visualization
Kaggle notebook environment	Execution of the recorded training and evaluation workflow
PyTorch	Embedding model, episodic optimization, and gradient calculation
torchvision	Pretrained EfficientNet-B0 and image transformations
NumPy	Numerical operations and handling prediction outputs
Pillow	Image loading, RGB conversion, and resizing
scikit-learn	Classification report and confusion matrix
Matplotlib	Training curves, evaluation plots, and image visualizations
OpenCV	Heatmap resizing, colour mapping, and overlay generation
Flask	Backend prediction interface
React	Image submission and result presentation

TorchScript and ONNX	Export formats for the embedding model

The notebook saves the checkpoint and serializes the encoder, fixed prototypes, labels, and preprocessing config. TorchScript and ONNX are export artefacts. Availability does not mean that both formats are used by the application or that they produce the same numeric output.

Precise dependency versions and hardware used cannot be provided by the existing notebook. They should be taken down from the development machine before final submission in order to allow reproduction. The specifics of application deployment and online testing are discussed in Section 5.5.

5.2 Component implementation

The classification part was tested with an embedding model using support based prototype computation. All parts from image visualization preparation to the estimation of the category and Grad-cam visualization are connected in one implementation.

5.2.1 Embedding model and episodic training

The Embedding Net keeps the convolutional feature extractor of Image Net-pretrained Efficient Net-B0 but replaces the classification head with adaptive average pooling, a linear projection from 1,280 to 512 features, and batch normalization. The backbone and projection are fine-tuned during training.

The episode sampler builds support and query sets from the images cached in the last section. For each class, eight support images and five query images are sampled. The functions compute prototypes averages the support embeddings per class and proctologist normalizes the query images and the support images and computes scaled-cosine similarity scores.

The loss function is cross-entropy as the model predicts a distribution over the set of possible documents based on the query and document labels. Adam optimizer learns rate and encoder at a rate set by a cosine annealing scheduler and learns rate for the learnable scale. Checks point which has maximal mean episodic validation accuracy.

5.2.2 Prediction of fixed-prototype

The build final prototypes function builds one prototype from each of the first 20 training images in the cache with the selected checkpoint. For an input image, the prediction function performs conversion to RGB, resize and normalization then outputs the embedding.

The embedding is then compared with the fixed prototypes. The resulting scores are then used to produce a predicted category, where the maximum is used. Softmax produces normalized class scores, but does not calibrate this data to be a probability of correctness.

5.2.3 Grad-CAM and model export

This is the class defining hooks on the encoder's last convolutional block. We backpropagate the scaled similarity value of the target category, spatially average the resulting gradient, weight the feature maps, rectify and normalize the resulting map, up-sample to image size and overlay it on the input image. Hooks are removed after visualization.

The model weights, Torch Script and ONNX encoders, fixed prototypes, labels, and preprocessing are exported by the notebook. An example of prediction with the exported Torch Script model is given in a manual. This demonstrates the example workflows but does not imply formal equivalence between export formats. Application integration and software testing are described in the following.

5.3 Integration with the other components

The germplasm classification component is integrated through the common web platform alongside land utilization analysis, WCLWD severity assessment, and advisory support. The React interface provides access to the classification function, while the Flask prediction service connects image submission with model inference and Grad-CAM generation.

For a classification request, the interface submits the image to the backend. The service applies the required preprocessing, generates an embedding, compares it with fixed class prototypes, and produces the predicted category and heatmap overlay. These

outputs are returned for display within the platform. The model weights, prototypes, labels, and preprocessing settings must remain consistent throughout this workflow.

Integration primarily provides shared access to the project's functions. It does not imply that the components share a trained model or that a predicted coconut category automatically determines planting recommendations, disease severity, or advisory responses. Such dependencies require explicitly implemented and tested data exchanges.

The notebook establishes the model-side prediction and visualization procedures, but it does not independently verify the complete interface-to-backend workflow or interactions with the other components. Application screenshots, API responses, and recorded integration tests are therefore required to substantiate end-to-end functionality. Integration testing is distinguished from episodic model validation, which evaluates a different prototype configuration.

5.4 Verification and software testing

Verification examined the prediction workflow, exported-model inference, and Grad-CAM generation using the examples recorded in the notebook. Application-level checks are listed separately where execution evidence is unavailable. Table 5.2 presents the test cases, expected outcomes, and recorded results.

Table 5.2: Test cases for the coconut classification component

ID	Test case	Expected result	Recorded result
TC1	Predict the selected King Coconut image using the original model and fixed prototypes	Output the King Coconut category	Demonstrated: King Coconut predicted
TC2	Predict the same image using the exported TorchScript encoder and fixed prototypes	Match the original model's predicted category	Demonstrated: same category and displayed scores; numerical

			equivalence not systematically tested
TC3	Generate Grad-CAM for the selected image	Produce a heatmap and overlay associated with the selected class score	Demonstrated: heatmap and overlay generated
TC4	Submit a non-image or corrupted file through the application	Reject the input with an appropriate error	Execution evidence unavailable
TC5	Submit a request without an image	Return a clear missing-input error	Execution evidence unavailable
TC6	Submit an image through the React interface	Display the returned category and Grad-CAM overlay	End-to-end test evidence unavailable
TC7	Measure prediction and visualization response time	Record response time in the target environment	No measured latency result available

The examples shown are of certain model functions, but do not necessarily represent the application's ability to work. The softmax score that is shown for an example is not taken as the probabilistic correctness of that example. Likewise, if a Grad-CAM overlay is generated, it does not imply that the highlighted regions are biologically relevant.

Additional validation must be performed using recorded application test cases to validate input, display, processing failure, and operational performance. Each piece of data must be backed up by either a log or screenshot before reporting as passed.

5.5 Deployment and operation

The trained module is ready for use in an application by exporting the encoder, static class prototypes, category labels, and preprocessing configuration. This notebook

shows how to load the Torch Script encoder and make a prediction for one image. This creates an inference script that can be run, but does not create production deployment or service availability.

Model artefacts should be loaded as a stable set, and the input pre-processed to match the training configuration. The classification is performed using the model in evaluation mode. The Grad-CAM also requires a gradient to be calculated through the chosen encoder layers without back-propagating through the model. The inference is executed in the application architecture by the Flask service.

On the operational side, there will need to be treated the invalid upload, failures to process, missing model files, etc. Also note down hosting setup, dependency versions, storage setup, and model refresh strategy. Measure the prediction and visualization response time in the target environment to claim performance.

There is no data available to justify a production hosting environment, concurrent-user capacity, uptime, or security-tested operation. The deployment is therefore identified as additional setup for application inference as production readiness will need documented end-to-end and operational testing.

6 6 RESULTS AND VALIDATION

6.1 6.1 Research results

The research results describe the training behavior and classification performance of the proposed component for Dwarf, King Coconut, and Tall.

6.1.1 Training Behavior

Figure 6.1 Training loss, training query accuracy, and validation query accuracy versus epoch for 40 epochs. Training loss decreased to near-zero while training query accuracy was able to reach near-100%. Validation query accuracy fluctuated and improved over epochs, but was below training accuracy, indicating overfitting. The random sampling of support and query images between training episodes also contributed to the fluctuations.

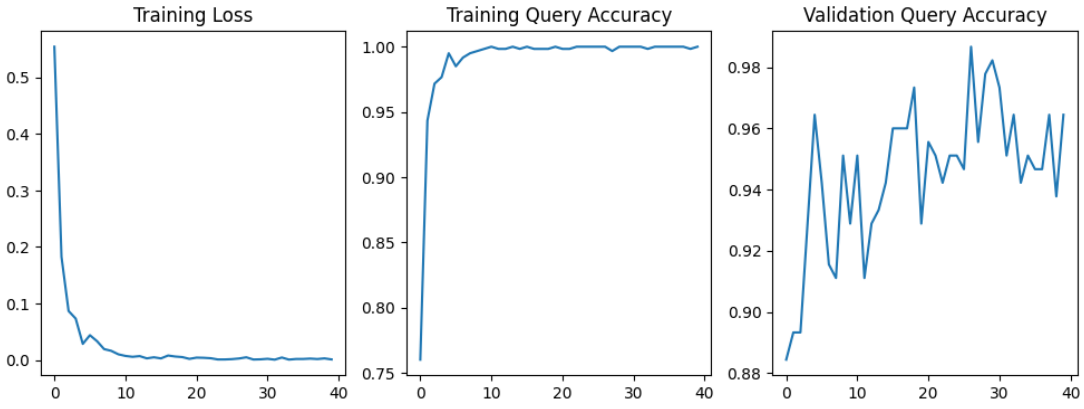


Figure 6.1:Training loss, training query accuracy, and validation query accuracy across 40 epochs.

The checkpoint with the highest validation-episode accuracy was selected for subsequent evaluation. The curves describe progress during training, while the final classification results were obtained through a separate evaluation of the selected checkpoint.

6.1.2 Classification Performance

The selected checkpoint was evaluated over 50 episodes. Each episode contained three categories, eight support images per category, and five query images per category. This produced 750 query predictions, with 250 predictions for each category. Of these, 724 were correct and 26 were incorrect, giving an overall accuracy of 96.53%.

Table 6.1 presents the classification report. Precision, recall, and F1-score are reported to two decimal places.

Table 6.1: Classification performance across 50 validation episodes

Category	Precision	Recall	F1-score	Query predictions
Dwarf	0.94	0.98	0.96	250
King Coconut	0.98	1.00	0.99	250
Tall	0.97	0.92	0.95	250
Macro average	0.97	0.97	0.97	750
Weighted average	0.97	0.97	0.97	750

King Coconut had the best individual F1-score, while Tall had the worst recall. Dwarf had the worst precision, because a few images from other classes were classified as Dwarf. Dwarf is listed in the results with no decimal places, so remember that the King Coconut recall of 1.00 is rounded and not evidence for perfect accuracy.

Figure 6.2. Confusion matrix with the number of correct predictions and errors between the three categories. It corresponds to the classification report and shows which categories were confused at test stage.

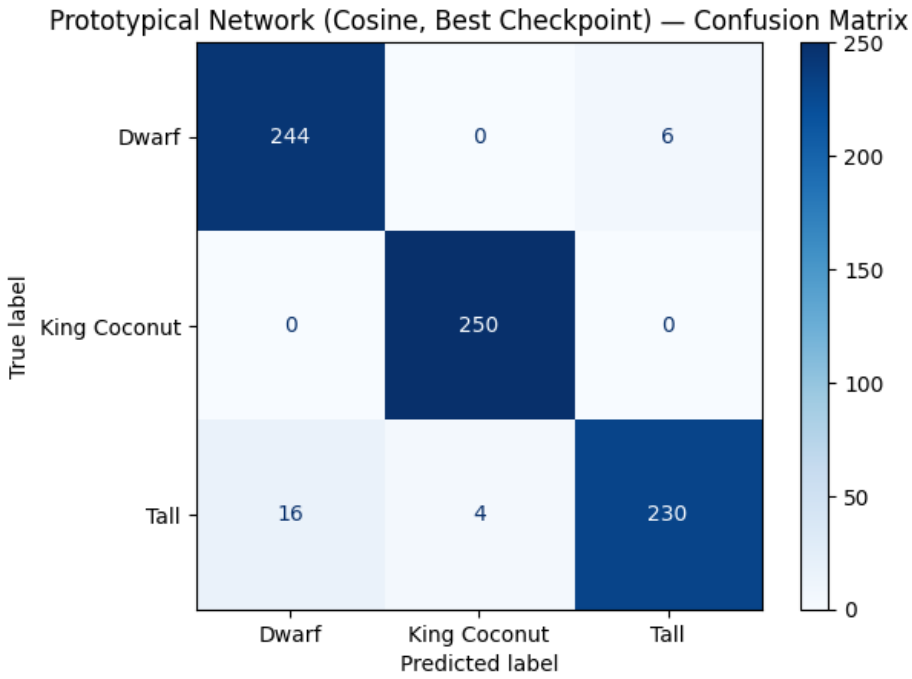


Figure 6.2: Confusion matrix for query predictions across 50 validation episodes.

6.1.3 Scope of the Evaluation

The results show that the classification performance appears promising given the validation dataset available. Nevertheless, the 750 predictions were obtained from 87 validation images, which means that these are repetitions from the same validation pool rather than from 750 separate images.

Consequently, this is an evaluation of known-category classification using the study's episodic procedure. It does not measure generalization to independent plantations, different image-acquisition conditions, or previously unencountered coconut categories. Additionally, the results reported here relate to the episode-specific support prototypes; performance with the set of fixed prototypes developed for application inference is a separate assessment.

6.2 6.2 Product and technical results

The implementation produced a classification component that generates a predicted coconut category and a corresponding Grad-CAM visualisation. The technical outputs also include trained weights, fixed class prototypes, label mappings, preprocessing settings, and exported inference models.

6.2.1 Classification Output

We generated fixed prototypes by calculating the mean embedding of 20 training images per class. We extracted scaled cosine similarity scores between an input image and the prototypes for classification.

Figure 6.3. An example of an input image which was misclassified as King Coconut. Fronds, overlapping structures, and orange coconut fruits are visible in this input image. This example illustrates that the inference pipeline generates a category label for an input image, but a single example does not indicate performance on other images or conditions in the field.



Figure 6.3: Example image classified as King Coconut by the implemented component.

6.2.2 Grad-CAM Visualization Output

6.4 King Coconut prediction original image on the top, then its corresponding Grad-CAM overlay heatmap below. The warmer the colour, the stronger the positive contribution to the score of the chosen classification, and the cooler the colour, the weaker.

The most prominent highlighted region can be seen in the right portion of the image, across the visible orange coconut fruits. Other highlighted areas can be seen around the stems at the bottom center. This gives some sense of a visual guide to look at the regions that are correlated with the prediction.

Additionally, the areas also include unshared parts of the original image and the image edge. We can therefore cannot confirm that the model only depends on the observable fruit coconut features or other biologically relevant features. We applied Grad-CAM to qualitatively examine the model – not as evidence of accurate identification [3], [8].

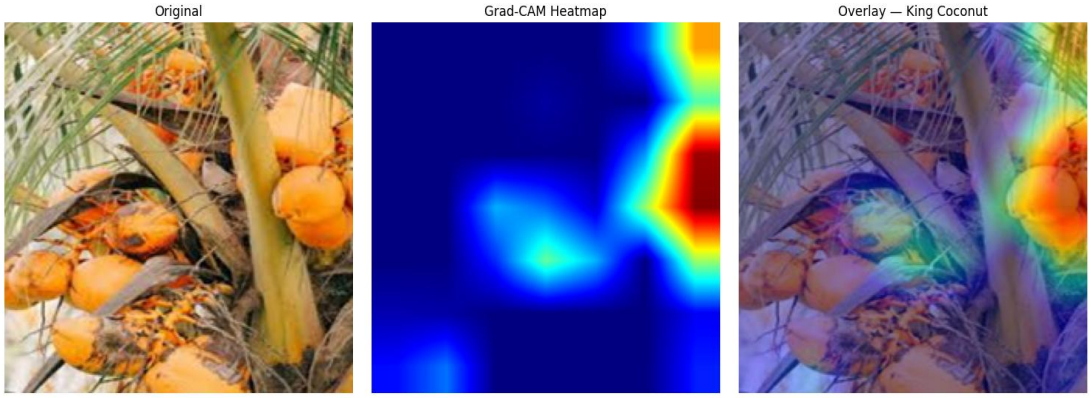


Figure 6.4: Original image, Grad-CAM heatmap, and overlay for the King Coconut prediction.

6.2.3 Model Export and Technical Verification

The trained component was saved with the supporting artefacts required for consistent inference. Table 6.2 summarizes these outputs.

Artefact	**Purpose**
Model checkpoints and weights	Preserve the selected trained parameters
TorchScript encoder	Provide an exported feature-extraction model
ONNX encoder	Support inference through compatible runtimes
Fixed class prototypes	Provide reference embeddings for the three categories
Label mapping	Associate output indices with category names

Preprocessing configuration	Record the input settings required for inference

The original model and the exported TorchScript model returned the same predicted category, with matching displayed scores for the manually inspected image, giving an early consistency check. Further numerical comparison across many images, and runtime validation of the ONNX export, are still to be done.

The 96.53% episodic accuracy as reported in Section 6.1 was achieved by employing support prototypes specific to individual episodes. Evaluation of the fixed prototypes defined for use in inference is reported separately.

6.3 User or stakeholder validation

Specific users of the component are farmers, plantation managers, and researchers. They need pre-analysis data of different coconut groups and categories. If the user validation could check whether the aforementioned types of users are able to upload images, interpret label prediction and visualization of Grad-CAM, and identify critical areas to seek expert advice or not.

This thesis evaluation has been centered on models and technical results. We have not carried out a formalized end-user study with documented users, usability tasks, questionnaires or expert judgement of the predictions and heatmaps reported here. We therefore do not make any numerical claims about user satisfaction, usability, stakeholder acceptance or decision improvements.

Next validation steps should include representative users performing image-upload and result-interpretation tasks. All the correctly and incorrectly classified examples from coconut experts could be scrutinized as to whether the expert agrees with the identified image regions. This would give an indication of the component's usefulness for stakeholders in field conditions.

6.4 Integration results

The germplasm classification component was developed to be seamlessly integrated into the common web platform using a React interface and Flask inference server. The ideal flow follows the image upload and preprocessing step, then the classification

prediction using a set of fixed prototypes, and then the output of the predicted category and overlayed with its Grad-CAM.

The outputs demonstrated above prove that the classification and visualization pipeline is capable of generating results for an input image. The notebook outputs, by themselves, do not prove that the whole browser-to-backend pipeline is functioning successfully. The aforementioned evidence does not show any integration-test results across services for image submission, receiving responses, or storing prediction history, or failures across services.

Thus, the integration with the rest of the project must be checked independently, considering the entire user workflow and its effect over the rest of the modules. The accuracy of the individual model can not be interpreted as the performance of the integrated system.

6.5 Summary of findings

The EfficientNet-B0 and prototype-based classifier scored 96.53% accuracy, with 724 correct predictions for 750 query predictions in 50 validation episodes. The highest F1-score was reported for King Coconut, while the lowest recall was obtained for Tall. The training curves converged, but remained apart from each other by a large gap in training and validation accuracy.

The implementation also returned predicted category labels, the Grad-CAM image overlays, and exported model artefacts. The prediction and visualization were confirmed by a manually inspected example, and an initial Torch Script check returned matching displayed output.

This indicates that a pre-testing visual classification analysis is feasible with this dataset, but is limited by multiple validations of the same images, selection of the validation images from the same pool for both training and testing, and episodic inference from a fixed set of class exemplars. Field validation, stakeholder validation, and integration testing are all needed for broader validation.

7 DISCUSSION

7.1 Interpretation of findings

These results imply that using an ImageNet-pretrained EfficientNetB0 feature extractor with prototype-based classification is capable of differentiating the Dwarf, King Coconut, and Tall image classes over the existing dataset, with a 96.53% accuracy, in terms of the learned embeddings separating well enough between support and query sets in the tested episodic setting. We note that it has not been tested against any other classifiers, and that this metric does not establish that the method is optimal.

Variations between the category-level results show that the overall classification was uneven. The highest F1-score was reported with King Coconut and the lowest with Tall, with the lowest recall with Tall. Although the unique fruit color could be a factor in detecting King Coconut, the effect of shape, colour, background, or other visual cues was not tested in the experiment, and so the reasons for these variations are unknown.

Training loss went to zero and training accuracy went to 100%, with validation accuracy being lower and fluctuating, indicating potential overfitting as well as fluctuation in episodic sampling. This illustrates the importance of testing the component on data collected independently before coming to a conclusion about how it performs on field images.

Grad-CAM gave a way of looking at the regions related to a prediction. The example shown revealed regions that were common to the coconut fruits and other parts of the plant. These overlaps lend themselves to qualitative inspection but do not prove that the model learned trustworthy morphological features; as discussed in [3], [8], take care when interpreting saliency maps.

In sum, results endorse the component as a visual sorting guide at the entry level for the three categories chosen. Its results do not infer and prove genetic identity, planting potential or extra productivity. Their usefulness needs to be judged from additional field testing, fixed-prototype inference, and stakeholder validation.

7.2 Comparison with literature baselines and existing solutions

The component uses Efficient Net for pretrained feature extraction and Prototypical Networks for support-based classification [1], [2]. The results show that this combination can be used for three coconut categories, but this application-specific verification does not verify recognition of unseen categories for which no data were available. This is because the same categories are present in training and validation.

Existing plant identification research has shown that deep learning performs well on both photographs of plants and herbarium specimens [5], [6], [10]. This study broadens the scope of application to consider the set of selected coconut categories. A direct comparison of accuracies isn't possible since the approaches use different data sets, definitions of categories, image collections and testing procedures.

The experiment did not include a controlled baseline comparison using the same split of the dataset and evaluation procedure. Therefore, it is not possible to conclude that the proposed method outperforms a standard classifier based on EfficientNet or other architectures, and so validate the advantage of the prototype-based classification.

The current applications Pl@ntNet and Seek [12], [13] are geared towards the identification of the wider plant/organism. [14], [12], [13] This section has a much more specialized purpose: to make a starting classification of Dwarf, King Coconut, Tall images with Grad-CAM visualizations. The scope is too specialized to generate better precision or ease of use than the two aforementioned applications, which were not tested on the study dataset.

Therefore, the main difference is the use of pretrained embeddings, prototype-based classification, and visualization of the selected categories. How well this work generalizes to coconut classes other than the ones tested here is yet to be seen.

7.3 Research contribution

The image-classification method for initial categorization of Dwarf, King Coconut and Tall from a small labelled coconut dataset. This work uses Image Net pretrained Efficient Net-B0 feature extractor, 512-dim embedding and prototype-based classification using scaled cosine similarity [1], [2].

Episodic evaluation, category-level metrics, a confusion matrix, and training curves are presented, giving application-related evidence. An accuracy of 96.53% was obtained which indicates that the method is viable for this evaluation data and procedure, but gives no proof of being better than other classifiers nor scalable to independent field conditions.

Grad-CAM provides a qualitative approach for identifying regions of the image associated with particular predictions [3]. For the present work, we use these visualizations as an interpretive tool and not as evidence of the correctness of classifications or as verification of identity.

Consequently, the contribution is an application of existing methods to a specific case of three chosen types of coconut rather than a new learning algorithm. It is thus a basis for later baseline comparisons, independent field testing, and expert evaluation.

7.4 Product contribution

Product contribution is an implemented component that transforms a coconut image into an initial category prediction and a corresponding Grad-CAM visualization. Integrates pre-processing, extraction of features, fixed-prototype classification and visual inspection into a single inference workflow for Dwarf, King Coconut and Tall.

The saved model weights, class prototypes, label mapping and preprocessing configuration are the artefacts that are required to be able to replicate the inference process and integrate with the common web platform. There are also options for subsequent deployment of exported model formats, which are subject to runtime verification.

The overall project will include this component along with information on land utilization, disease severity, and advice. The goal is to make identification easily available via an image upload process and give a visual output for the user or specialist to examine.

The contribution demonstrated is the working classification and visualization pipeline. No measurements of improvements in user decision-making, operational efficiency or coconut productivity were made. These benefits need to be tested by the user and in the field.

7.5 Limitations and threats to validity

7.5.1 Data set and generalizations

Among the images 428 total were taken at a single research plantation. Dwarf, King Coconut and Tall are not evenly represented. Several sources of variation between these images and those from other plantations are missing – backgrounds, lighting, palm age, device, and cultivation practices – so performance on other plantations is uncertain.

The partition is at image level, with no separation between palms. If there are images of the same palm in both parts, shared visual features will make validation simpler. This is a form of potential dependence in training and validation data, as opposed to true data leakage.

7.5.2 Evaluation Procedure

The last evaluation generated 750 query predictions by sampling the 87 validation images repeatedly. The 750 predictions are not independent samples of 750 different images. The same validation set was used for selecting the checkpoint as well as for final evaluation, so it is not an independent test estimate of the accuracy.

For training and validation, the same three categories were used. The experiment thus tests known-category classification rather than the recognition of new categories. Third, episodic evaluation in the same paradigm was based on support prototypes built per episode, whereas application inference uses constant prototypes. One cannot assume that the episodic accuracy we report is representative of the inference procedure we deploy.

A controlled baseline comparison or other repeated training runs with different seeds were not mentioned hence it is not known what real benefits can be contributed by the prototype-based classification and how stable the results are across training runs.

7.5.3 Explanation and Practical Validation

Grad-CAM visualizations were reviewed qualitatively without expert annotations and were not quantitatively scored. Regions of high heatmap activation may coincide with plant features while also present within the background or edge of the image. These

heatmaps are not definitive in terms of biological relevance or prediction accuracy as outlined in saliency studies [8].

This part of the component was limited to three categories, and it has not been tested to detect irrelevant images or otherwise unsupported categories of coconut. The softmax scores as shown are not calibrated and should not be read as "proven probabilities of correctness".

Expert stakeholder assessment, outside field tests, structured integration, latency, and export-equivalence evaluation were not part of the available evidence. These deficiencies prevent extrapolations concerning usability, operational dependability, and practical utility.

7.6 Implications

These results generate a foundation for the future exploration of pre-trained embeddings and prototype-based classification for coconut images. Subsequent experiments must compare the approach with standard classifiers using the same evaluation framework and check whether performance is consistent across training runs and on a third, independently collected dataset.

These features can provide initial category suggestions for farmers, plantation managers, and researchers. Predictions should be displayed as a form of preliminary visual identification only and provide appropriate instructions to indicate if specialist review may be necessary. The system should return an indication that it only covers Dwarf, King Coconut and Tall coconut when photographs outside this domain are provided.

While Grad-CAM provides a way of reviewing regions that contribute to a prediction, we need users to see just what the heatmaps mean. Expert review can assess whether the highlighted regions relate to valuable identification features or show when background influences the prediction.

In the broader context of the platform, category info should be associated with land use, disease evaluation, and guidance outputs. This information must be easy to analyze and use, and its significance should match the reliability of the different outputs. The research also supports further development and validation, leaving

refinements to cultivation practices, conservation measures, and yields to future research.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

The component enables accessible preliminary identification of Dwarf, King Coconut, and Tall varieties through an image-upload workflow. The system provides users with a predicted category and a Grad-CAM visualization for further inspection. On the shared platform, these outputs can be integrated with land utilization data, disease-severity assessments, and cultivation guidance.

Table 8.1 summarizes the intended users and potential benefits.

Table 8.1: Target users and proposed value of the component

Target user	Intended use	Potential value
Farmers	Obtain an initial category suggestion from a coconut image	Access preliminary identification information
Plantation managers	Inspect category predictions for photographed palms	Support initial documentation and referral for specialist assessment
Researchers	Review predictions, errors, and highlighted image regions	Support investigation of image-based coconut classification
Agricultural extension personnel	Demonstrate preliminary identification and explain its limitations	Support discussions with growers

The component's specialized scope and visualization output constitute its proposed value proposition. However, user demand, willingness to pay, time savings, and decision-making improvements have not yet been quantified. These anticipated benefits require evaluation through stakeholder interviews and pilot implementation.

Commercial positioning should describe the component as preliminary visual identification support. Specialist assessment is still necessary where reliable variety determination affects breeding, conservation, or planting decisions.

8.2 Market and Competitive Position

The component is introduced as a tool to undertake a basic initial visual classification of three coconut categories in an integrated coconut-management platform. The potential target group is farmers, plantation managers, researchers, and extension workers who need easy access to category information.

Current systems in Section 2.4, such as Pl@ntNet and Seek, offer broader identification capabilities [12], [13]. The proposed component primarily targets Dwarf, King Coconut, and Tall, and provides a Grad-CAM overlay that can be used to examine the parts of the image supporting its predicted output. Its integration with the general coconut-management platform adds another distinguishing factor.

Nevertheless, expertise by itself cannot provide an edge. No direct comparison between these applications on the coconut dataset, market-size estimation, price survey, or willingness-to-pay survey was made. Hence, the model's achieved accuracy cannot support the claim of commercial success.

A first step to adoption could be a pilot with plantation managers, researchers or extensionists. The pilot should look at identification needs, ease of use, acceptable levels of error, and the added value of combining category predictions with other platform outputs. This will inform whether the component should be maintained as a research tool, institutional tool, or commercial platform.

8.3 Business or Sustainability Model

The component could therefore be initially supported by an institutional model within the integrated coconut-management platform. An institution, such as a university, research organization, or agricultural partner, could support a limited pilot while the team determines field performance and user needs. No financial agreement or institution partnership is implied.

Long-term operation includes the costs of hosting, image storage, public maintenance, user help, and model assessment. Expanding the dataset and specialist review will add ongoing costs. These costs should be determined during pilot operation to inform a pricing/funding model.

Future alternatives may include grant-supported access, an institutional license, or subscription-based access for parts of the wider platform functions in plantation management. They are not, and should not be assumed to be, sources of income; these alternatives need to be tested for value and affordability.

Technical sustainability would be achieved through documented deployment procedures, explicit versioned model artifacts, and defined individuals responsible for updates. Archived models and prototypes would need to be tested prior to release, with prior versions held for traceability and rollback. This would enable the component to evolve with the rest of the platform.

8.4 Scalability, Intellectual Property and Compliance

8.4.1 Scalability

Increasing use demands analysis of inference time, memory loads, concurrent requests, and image-storage needs. Since Grad-CAM involves extra gradient calculations, its cost should be isolated from that of classification. No load-testing information has been provided so far, so it is impossible to know how many users the current implementation can support.

The prototype expansion would have to incorporate labeled images; the modified prototypes and classifiers should be changed accordingly, and the new classifier would have to be tested. The number of prototypes could be increased by simply adding them without guaranteeing recognition of a new class, as the feature extractor was only trained and tested on the previous three.

8.4.2 Intellectual Property

Project artefacts include application code, trained weights, class prototypes, configuration files, and research notes. Owner and use rights should be discussed with the project team, university and other data owners before releasing or selling the assets.

Licenses and terms of use of any software dependencies, pretrained weights or collected images should also be considered. There is no patent protection, exclusive rights to the dataset or commercial licensing agreement applied to this thesis.

8.4.3 Compliance and Responsible Operation

Prior to launching more broadly, the platform needs to determine the right data protocol for users' uploaded images, identifiers, and location data, such as why it is being collected, who can view it, how long to keep it, and how it is erased. The existing data protection requirements need to be evaluated.

The output should indicate that predictions are a rough visual classification of the three supported classes. Heatmaps and scores shown should be clearly explained, and consulting a specialist should be recommended where the identification has critical implications for cultivation, breeding or conservation.

Neither compliance nor operational readiness were evidenced by the model evaluation presented. Both need to be reviewed with respect to the deployed platform, its data management practices, and its purpose.

8.5 Alignment with Sustainable Development Goals

This component is probably relevant for SDG 2: Zero Hunger and SDG 9: Industry, Innovation and Infrastructure. The 2030 agenda of the United Nations links these goals to sustainable agriculture, preservation of cultivated-plant genetic diversity and scientific and technological advancement [15]. Department of Economic and Social Affairs

It has an indirect association with SDG 2. Basic information on coconut category can serve as a teaching and initial recording resource before the need for specialist evaluation. Visual classification can only provide indications of the genetic diversity, and this article did not record progress on conservation, food, and agricultural productivity.

The project further ties into SDG 9 the innovation aspect, of the SDGs, via the production and testing of an AI focused agricultural element. Its prediction pipeline,

its visualization and ability to produce model artefacts for export provide a test bed for further technical development and in-field testing.

These links show intended alignment and are not proof of SDG impact. Future evaluation should consider if field use enhances access to identification data and enables meaningful Agri- or research actions.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

We implemented and evaluated a computer-vision classification module to pre-detect 'Dwarf', 'King Coconut', and 'Tall' species. This pipeline leveraged a 512-D embedding space generated from an ImageNet-pretrained EfficientNet-B0 backbone classifier, a classifier based on feature space prototypes, and complementary regional heatmaps produced via Grad-CAM to visualize image regions corresponding to detections of specific interest.

On a dataset of 428 coconut images, the selected model achieved 96.53% accuracy, with 724 correct predictions out of 750 query predictions across 50 validation episodes. The results demonstrate the ability to differentiate the chosen classes in the study dataset and the episodic testing procedure. The implementation had also produced fixed class prototypes and exported model artefacts for use at inference.

The assessment has significant constraints. Query predictions relied on iterative sampling from 87 validation photos, and the similar validation pool was employed for checkpoint choice. We couldn't ascertain independent-subject generalization, fixed-prototype classification accuracy, or real-world applicability. The Grad-CAM final result will not validate biologically relevant logic or genetic id.

The program provides a practical, empirically tested basis for initial visual coconut identification in the wider coconut-management project. Further testing across plantations, comparison with other classifiers, and validation by experts and end users are required before deployment on a wider scale.

9.2 Achievement of objectives

The study achieved its core objective of developing and evaluating an image-classification component for preliminary identification of Dwarf, King Coconut, and Tall. Table 9.1 summarizes the achievements and the scope of the supporting evidence.

Table 9.1: Summary of objective achievement

Objective	Achievement and supporting evidence
Prepare labeled coconut images for model development	Organized 428 images into three categories, with 341 training images and 87 validation images
Develop the classification model	Implemented an EfficientNet-B0 feature extractor, 512-dimensional embeddings, and prototype-based classification
Evaluate classification performance	Obtained 96.53% accuracy across 50 validation episodes and reported category-level metrics and a confusion matrix
Provide visual inspection of predictions	Implemented Grad-CAM and generated heatmaps and overlays for selected predictions
Prepare the component for application inference	Saved fixed prototypes, label mappings, preprocessing settings, and exported model artifacts

The classification and visualization aims were met within the experimental scope. Application inference preparation was also carried out, but systematic evaluation of fixed-prototype performance and the full integrated workflow needs to be done.

The study did not demonstrate genetic identification, increased productivity, or performance under independent field conditions. These results are beyond what the reported experiments can support.

9.3 Individual research and product contributions

An image-classification approach to automate the initial screening of Dwarf, King Coconut and Tall. This comprised dataset generation, a transfer learning approach using an Image Net pre-trained Efficient Net-B0 encoder, prototype-based classification, and episodic evaluation. The investigation produces application-specific evidence, in the form of classification metrics, training curves, and a confusion matrix.

Individual product contribution is an inference pipeline that produces the predicted product category along with a Grad-CAM visualization. Fixed class prototypes, model weights, label maps, post-processing settings, and exported encoders were created to make the application inference-ready and compatible with the common web platform.

This contribution is only the germplasm-identification part of the larger project, the development of which is entirely beyond the scope of this thesis. What I have shown is, in part, for preliminary visual identification and examination, not for genetic authenticity or better horticultural results.

9.4 Future directions

Future work should also focus on independent evaluation using images sourced from other plantations, devices, and acquisition conditions. Tracking individual palm identifiers would allow the training and evaluation partitions to be split by palm, preventing potentially related images from appearing in both sets. A distinct test set should be maintained for model comparison.

Of course, we cannot validate the fixed-prototype inference pipeline this way; it must be assessed directly, as it differs from the episodic classification method we used to report accuracy. We can compare it more rigorously with traditional EfficientNet classification and other models in head-to-head training tests, repeated many times.

Additional tests should include rejecting images with other content and unsupported coconut categories. Evaluation of the score calibration and image quality checking should also inform when other images are requested or when specialist review is needed.

Grad-CAM evaluation could include coconut experts examining both wrong and right predictions. When appropriate annotations are present, regions of interest could be cross-referenced to other plant parts. This would provide more convincing evidence of the visualizations' value.

Finally, the entire application flow should be integrated, response times tested, and usability validated with representative users. A pilot deployment in a production environment would allow refinement of real-world utility, maintenance, and operating

costs prior to deployment at scale. Additional categories could be added once sufficient labeled data was collected, and the model was refined.

10 REFERENCES

[1] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in Proc. 36th Int. Conf. Machine Learning, vol. 97, 2019, pp. 6105–6114. [Online].

[2] J. Snell, K. Swersky, and R. S. Zemel, “Prototypical networks for few-shot learning,” in Advances in Neural Information Processing Systems, vol. 30, 2017. [Online].

[3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in Proc. IEEE Int. Conf. Computer Vision (ICCV), 2017, pp. 618–626. [Online].

[4] G. K. Ekanayake, S. A. C. N. Perera, P. N. Dassanayake, and J. M. D. T. Everard, “Varietal classification of new coconut (Cocos nucifera L.) forms identified from Southern Sri Lanka,” COCOS, vol. 19, no. 1, pp. 41–50, 2012, doi: 10.4038/cocos.v19i1.4749.

[5] S. H. Lee, C. S. Chan, P. Wilkin, and P. Remagnino, “Deep-plant: Plant identification with convolutional neural networks,” in Proc. IEEE Int. Conf. Image Processing (ICIP), 2015, pp. 452–456, doi: 10.1109/ICIP.2015.7350839.

[6] J. Carranza-Rojas, H. Goëau, P. Bonnet, E. Mata-Montero, and A. Joly, “Going deeper in the automated identification of herbarium specimens,” BMC Evolutionary Biology, vol. 17, Art. no. 181, 2017, doi: 10.1186/s12862-017-1014-z.

[7] S. P. Mohanty, D. P. Hughes, and M. Salathé, “Using deep learning for image-based plant disease detection,” Frontiers in Plant Science, vol. 7, Art. no. 1419, 2016, doi: 10.3389/fpls.2016.01419.

[8] J. Adebayo, J. Gilmer, M. Muelly, I. Goodfellow, M. Hardt, and B. Kim, “Sanity checks for saliency maps,” in Advances in Neural Information Processing Systems, vol. 31, 2018.

[9] L. C. J. Kamara, S. A. C. N. Perera, and P. N. Dassanayaka, “Sri Lanka yellow semi tall; a new addition to the coconut (Cocos nucifera L.) classification in Sri Lanka,” COCOS, vol. 22, no. 1, pp. 49–55, 2016, doi: 10.4038/cocos.v22i1.5811.

[10] Y. Sun, Y. Liu, G. Wang, and H. Zhang, “Deep learning for plant identification in natural environment,” Computational Intelligence and Neuroscience, vol. 2017, Art. no. 7361042, 2017

[11] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, “How transferable are features in deep neural networks?” in Advances in Neural Information Processing Systems, vol. 27, 2014, pp. 3320–3328.

[12] Pl@ntNet, “Pl@ntNet’s main features,” Pl@ntNet Documentation. Accessed: Oct. 10, 2026. [Online].

[13] iNaturalist, “Seek by iNaturalist.” Accessed: Oct. 10, 2026. [Online].

[14] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” Journal of Management Information Systems, vol. 24, no. 3, pp. 45–77, 2007, doi: 10.2753/MIS0742-1222240302.

[15] United Nations General Assembly, “Transforming our world: The 2030 Agenda for Sustainable Development,” Resolution A/RES/70/1, 2015. [Online].

11 GLOSSARY

Term	Definition
Class prototype	A reference vector formed by averaging the embeddings of support images belonging to a category.
Cosine similarity	A measure of similarity between vectors based on the angle between them.
Embedding	A numerical representation of an image used to compare its visual features with those of other images.
Episodic training	Training organized into tasks containing support images and query images.
Feature extractor	The part of a model that converts an image into a numerical representation of its visual characteristics.
Few-shot learning	A learning approach that investigates classification using a small number of labeled examples per category.
Fixed prototypes	Class reference vectors constructed in advance and reused during application inference.
Germplasm	Genetic material maintained or used for conservation, breeding, and research.
Grad-CAM	Gradient-weighted Class Activation Mapping; a method that highlights image regions associated with a selected model output.
ImageNet pretraining	Initial training on the ImageNet dataset before adapting a model to another task.
Inference	Applying a trained model to an input image to generate a prediction.

Preliminary visual identification	An initial category suggestion based on visible image characteristics, requiring further assessment where necessary.
Prototypical Network	A model that classifies query images by comparing their embeddings with class prototypes.
Query set	Images classified against prototypes within an episode.
Scaled cosine similarity	Cosine similarity multiplied by a scale parameter to produce classification scores.
Softmax	A function that converts class scores into positive values that sum to one; these values are not necessarily calibrated probabilities.
Support set	Labeled images used to construct class prototypes within an episode.
Transfer learning	Adapting knowledge learned from one task or dataset to another.

12 APPENDICES

12.1 Appendix A Research instruments or extended results

12.1.1 A.1 Per-Epoch Training Log

Column A.1 displays the mean training loss, training query accuracy, and validation query accuracy over the course of 40 epochs. We computed training and validation accuracy as the average across 40 and 15 episodes per epoch, respectively. All accuracy values are represented as a proportion.

An asterisk indicates an epoch where the validation accuracy was greater than the previous best accuracy and a model checkpoint was saved. The final selected checkpoint was at epoch 27, with a mean validation query accuracy of 98.67%. The selected checkpoint was not the same as the 50-episode validation with 96.53% accuracy.

Table 12.1:Per-epoch training log

Epoch	Training loss	Training query accuracy	Validation query accuracy	Learning rate
1*	0.5546	0.7600	0.8844	10.0×10^{-5}
2*	0.1826	0.9433	0.8933	9.9×10^{-5}
3	0.0867	0.9717	0.8933	9.9×10^{-5}
4*	0.0733	0.9767	0.9289	9.8×10^{-5}
5*	0.0285	0.9950	0.9644	9.6×10^{-5}
6	0.0439	0.9850	0.9422	9.5×10^{-5}
7	0.0333	0.9917	0.9156	9.3×10^{-5}
8	0.0192	0.9950	0.9111	9.0×10^{-5}
9	0.0162	0.9967	0.9511	8.8×10^{-5}
10	0.0100	0.9983	0.9289	8.5×10^{-5}
11	0.0072	1.0000	0.9511	8.2×10^{-5}

12	0.0056	0.9983	0.9111	7.9×10^{-5}
13	0.0069	0.9983	0.9289	7.6×10^{-5}
14	0.0029	1.0000	0.9333	7.3×10^{-5}
15	0.0047	0.9983	0.9422	6.9×10^{-5}
16	0.0028	1.0000	0.9600	6.5×10^{-5}
17	0.0079	0.9983	0.9600	6.2×10^{-5}
18	0.0061	0.9983	0.9600	5.8×10^{-5}
19*	0.0051	0.9983	0.9733	5.4×10^{-5}
20	0.0020	1.0000	0.9289	5.0×10^{-5}
21	0.0042	0.9983	0.9556	4.6×10^{-5}
22	0.0038	0.9983	0.9511	4.2×10^{-5}
23	0.0031	1.0000	0.9422	3.8×10^{-5}
24	0.0009	1.0000	0.9511	3.5×10^{-5}
25	0.0009	1.0000	0.9511	3.1×10^{-5}
26	0.0015	1.0000	0.9467	2.7×10^{-5}
27*	0.0027	1.0000	0.9867	2.4×10^{-5}
28	0.0047	0.9967	0.9556	2.1×10^{-5}
29	0.0008	1.0000	0.9778	1.8×10^{-5}
30	0.0011	1.0000	0.9822	1.5×10^{-5}
31	0.0021	1.0000	0.9733	1.2×10^{-5}
32	0.0005	1.0000	0.9511	1.0×10^{-5}
33	0.0044	0.9983	0.9644	0.7×10^{-5}
34	0.0007	1.0000	0.9422	0.5×10^{-5}
35	0.0017	1.0000	0.9511	0.4×10^{-5}

36	0.0018	1.0000	0.9467	0.2×10^{-5}
37	0.0024	1.0000	0.9467	0.1×10^{-5}
38	0.0017	1.0000	0.9644	0.1×10^{-5}
39	0.0028	0.9983	0.9378	0.0×10^{-5}
40	0.0010	1.0000	0.9644	0.0×10^{-5}

A.2 Grad-CAM for the Prototype-Similarity Score

Below is a modified excerpt showing the base Grad-CAM algorithm we apply to explain the scaled cosine-similarity score between a query embedding and the target class prototype [3]. It assumes one input image and that hooks have been registered on the convolutional layer of choice for capturing the activations and the gradients.

Although the scale parameter is named log_scale, the implementation multiplies it directly by cosine similarity without exponentiation.

```
def generate(self, input_tensor, prototypes, log_scale,
target_class_idx=None):
    self.model.eval()
    input_tensor = input_tensor.clone().requires_grad_(True)
    emb = self.model(input_tensor)                        # 1 x 512 embedding
    q_norm = F.normalize(emb, dim=1)
    p_norm = F.normalize(prototypes, dim=1)
    logits = log_scale * (q_norm @ p_norm.t())            # scaled cosine
similarity
    if target_class_idx is None:
        target_class_idx = logits.argmax(dim=1).item()
    score = logits[0, target_class_idx]
    self.model.zero_grad()
    score.backward()                                      # gradients via
hooks
    gradients = self.gradients[0]                         # C x H x W
    activations = self.activations[0]                     # C x H x W
    weights = gradients.mean(dim=(1, 2))                  # alpha_m (Eq. 3.4)
    cam = (weights[:, None, None] * activations).sum(0)   # weighted sum
    cam = F.relu(cam)                                     # Eq. 3.5
    cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
    probs = F.softmax(logits, dim=1).detach().cpu().numpy()[0]
    return cam.cpu().numpy(), target_class_idx, probs
```

The excerpt uses a vectorized weighted sum over channels, corresponding to the explicit channel loop in the notebook implementation. The resulting map is subsequently resized and overlaid on the original image. It highlights regions associated with the selected score and does not establish biological validity or prediction correctness.